

THE ART OF GAME DEVELOPMENT WITH UNITY



HARIOM JAGTAP

The Art of Game Development

With Unity

.....By Hariom.M.Jagtap.

OceanofPDF.com

The Art Of Game Development With Unity

-By Hariom Jagtap

Author

Prof. Hariom. Jagtap

Email: hariomjagtap8@gmail.com

Mobile No.: +91 9284155227

Publisher

Mahenjali Creations

Printer

Star Copiers Pvt. Ltd.

Kumthekar Rd, Sadashiv Peth, Pune,

Maharashtra 411030

First Edition: 22/7/2024

ISBN: 978-93-340-8442-9



Pages: 92

Price: 250/-

*Crafting worlds with code and art,
Where dreams and pixels intertwine,
In every game, a journey starts.*

Hariom Jagtap

Summary

‘The Art Of Game Development with Unity Engine" by Hariom Jagtap serves as an Guide for those who are new to game development and at intermediate level using the Unity engine. The book covers the fundamental aspects of Unity, providing readers with the knowledge needed to start creating their own games. The book contains numerous examples and information to help readers gain confidence in using Unity. By the end of the book readers should be able to understand the core components of the Unity engine.

Table of Content ’ s

1. Introduction

- History Of Game
- **What is Unity?**
- **Why Choose Unity for Game Development?**

2. Setting Up Unity

- System Requirements
- **Installing Unity Hub**
- **Installing Unity Editor**

3. Unity Basics

- Unity Interface Overview
- **Key Components (Game Objects, Components, Scenes)**

4. Creating Your First Project

- Importing Asset From 3D Software
- **Setting Up a New Project**
- **Understanding the Project Structure**
- **Setting up The Arena**

5. Lights, Cameras, and Shadow

- Working with cameras
- Orientating your frustum
- Creating a Skybox
- Working with lighting
- Understanding shadows

6. Basic Scripting with C#

- Concept Of OOP's
- **Introduction to C# for Unity**
- **Creating and Attaching Scripts**
- **Basic Scripting Concepts**

7. Working with Assets Store

- Using the Asset Store

8. Basic Game Mechanics

- Creating and Moving Game Objects
- **Handling User Input**
- **Basic Physics and Collisions**

9. Building and Running Your Game

- Configuring Build Settings
- **Building for Different Platforms**
- **Building your application**
- **Running and Testing Your Game**

10.

FPS and TPS Concepts

- First Person Shooter (FPS)
- **Third Person Shooter (TPS)**

11.

Next Steps and Resources

- Learning Resources
- **Community and Support**

OceanofPDF.com

Chapter 1: Introduction

History Of Game

The history of games spans thousands of years, evolving from simple board games and physical contests to complex digital and video games. Here is an overview of the significant developments in the history of games:

Ancient Games

1. Board Games:

- **Senet:** One of the oldest known board games, originating in ancient Egypt around 3100 BCE.
- **Go:** Originated in China over 2,500 years ago and remains popular today.
- **Mancala:** Played in Africa for thousands of years, involving the movement of stones or seeds across a board with pits.

2. Dice Games:

- Dice made from bones, stones, and other materials have been found in archaeological sites dating back to around 3000 BCE in ancient Mesopotamia.

3. Physical Games:

- **Olympic Games:** First held in ancient Greece in 776 BCE, featuring various athletic contests.

Medieval to Early Modern Period

1. Chess:

- Originated in India around the 6th century CE as Chaturanga, evolving into modern chess as it spread to Persia, the Islamic world, and Europe.

2. Playing Cards:

- Likely originated in China during the Tang Dynasty (618-907 CE) and spread to Europe by the 14th century.

3. **Dominoes:**

- Also believed to have originated in China around the 12th century.

19th Century

1. **Board Games:**

- **The Mansion of Happiness** (1843): Considered one of the first commercially published board games in the United States.
- **Monopoly**: Created in the early 20th century and became one of the most famous board games globally.

2. **Outdoor and Parlor Games:**

- Games like croquet, lawn tennis, and various parlor games became popular during the Victorian era.

20th Century

1. **Card Games:**

- **Bridge** and **Poker**: Became highly popular, with many variations and tournaments.

2. **Early Mechanical and Electronic Games:**

- **Pinball**: Developed in the 1930s.
- **Computer Games**: The first computer-based games emerged in the 1950s and 1960s, such as "Tennis for Two" (1958) and "Spacewar!!" (1962).

3. **Video Games:**

- **Arcade Games**: The 1970s saw the rise of arcade games like "Pong" (1972) and "Space Invaders" (1978).

- **Home Consoles:** The first home video game consoles, such as the Atari 2600, were released in the late 1970s and early 1980s.
- **Personal Computers:** The 1980s and 1990s saw the growth of computer games with titles like "Pac-Man," "Tetris," and "Doom."

21st Century

1. Advanced Video Games:

- **Console and PC Games:** Significant advancements in graphics, gameplay, and storytelling with consoles like PlayStation, Xbox, and PC gaming.
- **Online and Multiplayer Games:** The rise of MMORPGs (Massively Multiplayer Online Role-Playing Games) like "World of Warcraft" and battle royale games like "Fortnite."

2. Mobile Games:

- The proliferation of smartphones led to a boom in mobile gaming with games like "Angry Birds," "Candy Crush," and "Pokémon GO."

3. Virtual Reality (VR) and Augmented Reality (AR):

- Innovations in VR and AR have created immersive gaming experiences, such as "Beat Saber" and "Half-Life: Alyx."

4. Esports:

- Competitive gaming has become a global phenomenon, with professional players and teams competing in games like "League of Legends," "Dota 2," and "Counter-Strike."

The history of games reflects human creativity and the desire for entertainment, challenge, and social interaction. As technology continues to

advance, the future of gaming promises even more exciting developments and experiences

What is a Game?

A Game is an software that involves interaction with a user interface to generate visual feedback on a video device such as a Computer Monitor.

A Game is a structured form of play, usually undertaken for entertainment or fun, and sometimes used as an educational tool. Games are distinct from work, which is usually carried out for remuneration, and from art, which is more concerned with the expression of ideas. However, the distinction is not clear-cut, and many games also consider work (such as professional sports players) or art (such as jigsaw puzzles or games involving artistic layout such as Mahjong, solitaire, or some video games).

Types of Game Genres

There are several different game genres, each offering unique experiences and challenges. Here are some of the most popular ones:

1. **Action:** Fast-paced games that require quick reflexes and hand-eye coordination. Examples include platformers, shooters, and fighting games.
2. **Adventure:** Games that focus on exploration and narrative, often involving puzzle-solving and item collection. Examples include point-and-click adventures and text-based games.
3. **Role-Playing Games (RPGs):** Games where players control a character or party of characters, often involving complex storylines and character development. Examples include turn-based RPGs and action RPGs.
4. **Simulation:** Games that simulate real-world activities, such as flying a plane, managing a city, or farming. Examples include flight simulators, city builders, and life simulation games.
5. **Strategy:** Games that require players to plan and make decisions to achieve victory. Examples include real-time strategy (RTS) and turn-based strategy games.

6. **Sports:** Games that simulate playing sports, from traditional sports like soccer and basketball to extreme sports like skateboarding and snowboarding.
7. **Puzzle:** Games that challenge the player's problem-solving skills, often involving logic, pattern recognition, and strategy.
8. **Horror:** Games designed to scare the player, often featuring dark atmospheres, suspense, and a sense of dread.

Popular Game Development Companies

Several companies have made significant contributions to the game development industry. Here are a few notable ones:

- **Nintendo:** Known for creating iconic franchises like Mario, Zelda, and Pokémon.
- **Sony Interactive Entertainment:** Famous for the PlayStation consoles and exclusive games like The Last of Us and God of War.
- **Microsoft Studios:** Known for the Xbox consoles and franchises like Halo and Forza.
- **Ubisoft:** Known for series like Assassin's Creed, Far Cry, and Watch Dogs.
- **Electronic Arts (EA):** Known for sports games like FIFA and Madden NFL, as well as franchises like The Sims and Battlefield.
- **Valve:** Known for games like Half-Life, Portal, and the Steam platform.

What is Unity?

Unity is a powerful and flexible game engine that allows developers to create video games for various platforms, including PCs, consoles, mobile devices, and the web. It provides a suite of tools and features that simplify the game development process, enabling developers to focus more on creativity and less on technical complexities.

Why Choose Unity for Game Development?

Unity is favoured by many game developers for several reasons. First, it supports multiple platforms, meaning you can develop a game once and deploy it across different devices and operating systems. Second, Unity offers extensive documentation and tutorials, making it easier for beginners to learn and for experts to find solutions to specific problems. Third, Unity has a large and active community, providing a network of support and collaboration opportunities. Finally, Unity ' s Asset Store provides a vast array of pre-made assets, which can significantly speed up development.

Chapter 2: Setting Up Unity

System Requirements

Before installing Unity, ensure your computer meets the minimum system requirements. For Windows, you need Windows 7 SP1 or later, 64-bit versions only. For Mac, you need macOS 10.12 or later. For Linux, Unity supports Ubuntu 16.04, Ubuntu 18.04, and CentOS 7.

Installing Unity Hub

To get started with Unity, you first need to install Unity Hub, a management tool for Unity projects and installations. Visit the [Unity website](#) and download Unity Hub. Follow the installation instructions for your operating system to complete the setup.

Installing Unity Editor

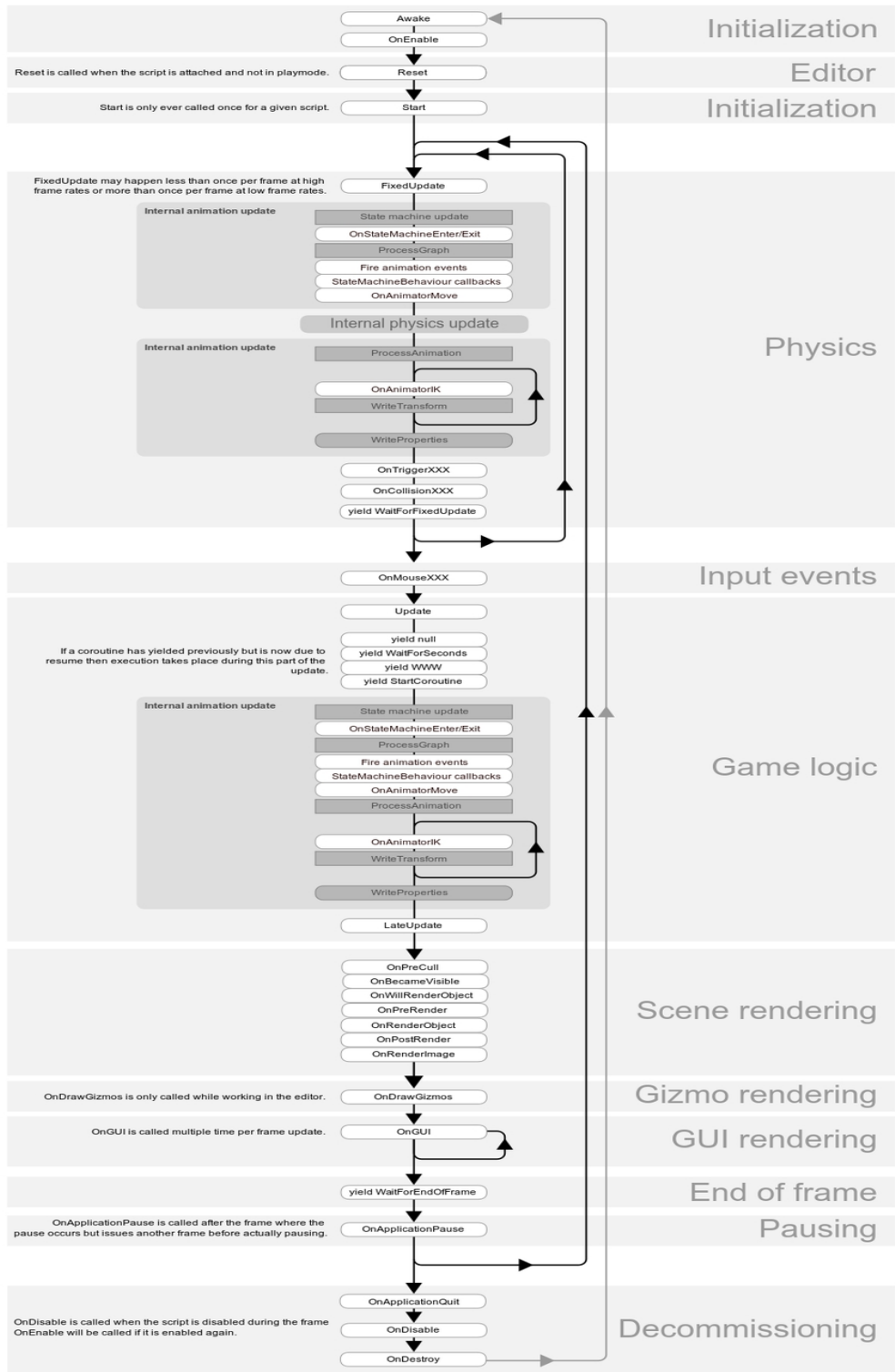
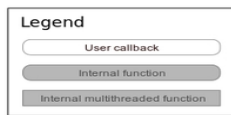
Once Unity Hub is installed, open it and click on the "Installs" tab. Click the "Add" button to install a new version of the Unity Editor. You can choose the version of Unity you want to install and select additional modules for your target platforms, such as Windows Build Support or Android Build Support.

Chapter 3: Unity Basics

Unity Script lifecycle

The diagram below summarizes how Unity orders and repeats event functions over a script ' s lifetime.

Unity Script lifecycle Flowchart/Unity Event Structure



- **Awake:** First lifecycle function called when a new instance of an object is created. Always called before any Start functions. If a GameObject is inactive during start up, Awake is not called until it is made active.
- **OnEnable:** Called when the object becomes enabled and active, always after Awake (on the same object) and before any Start.
- **Reset:** called to initialize the script 's properties when it is first attached to an object and also when the Reset command is used.
- **Start:** called before the first frame update only if the script instance is enabled.
- **FixedUpdate :** FixedUpdate happens at fixed intervals of in-game time rather than per frame. Since these updates are fixed and frame rate is variable, there may be no fixed update during a frame when frame rate is high, or multiple fixed updates per frame when frame rate is low. All physics calculations and updates occur immediately after FixedUpdate and because it 's frame-rate independent you don 't need to multiply values by Time.deltaTime when calculating movement in a FixedUpdate.
- **Update :** Update is called once per frame and is the main function for frame updates.
- **LateUpdate :** LateUpdate is called once per frame, after Update has finished. Any calculations performed in Update will have completed when LateUpdate begins. A common use for LateUpdate would be a following third-person camera. If you make your character move and turn inside Update, you can perform all camera movement and rotation calculations in LateUpdate. This will ensure that the character has moved completely before the camera tracks its position.
- **OnGUI:** OnGUI is called multiple times per frame in response to GUI events. The Layout and Repaint events are processed first, followed by a Layout and keyboard/mouse event for each input event.
- **OnApplicationPause :** If the game is hidden (fully or partly) by another application then it pauses and Unity calls OnApplicationPause with true . When the game regains focus, Unity calls OnApplicationPause with false .

- **OnApplicationQuit:** OnApplicationQuit is called on all game objects before the application is quit. In the editor it is called when the user stops playmode.
- **OnDisable:** OnDisable is called when the behaviour becomes disabled or inactive.
- **OnDestroy:** OnDestroy is called after all frame updates for the last frame of the object 's existence (the object might be destroyed in response to Object.Destroy or at the closure of a scene).

☐ **Time.deltaTime:**

1. Time.deltaTime is a completion time in seconds since the last frame.
2. Time.deltaTime is a Unity specific variable that represents the time interval in seconds it took from the last frame to the current frame. It measures the interval between the current frame and the last one.

```
using UnityEngine;
// Rotate around the z axis at a constant speed
public class ConstantRotation : MonoBehaviour
{
    public float degreesPerSecond = 2.0f;
    void Update()
    {
        transform.Rotate(0, 0, degreesPerSecond * Time.deltaTime);
    }
}
```

☐ **Time.time :**

1. Time.time elope time since the project started playing.
2. The time at the beginning of the current frame in seconds since the start of the application.

```

//If the Fire1 button is pressed, a projectile
//will be Instantiated every 0.5 seconds.

using UnityEngine;
using System.Collections;

public class Example : MonoBehaviour
{
    public GameObject projectile;
    public float fireRate = 0.5f;
    private float nextFire = 0.0f;

    void Update()
    {
        if (Input.GetButton("Fire1") && Time.time > nextFire)
        {
            nextFire = Time.time + fireRate;
            Instantiate(projectile, transform.position, transform.rotation);
        }
    }
}

```

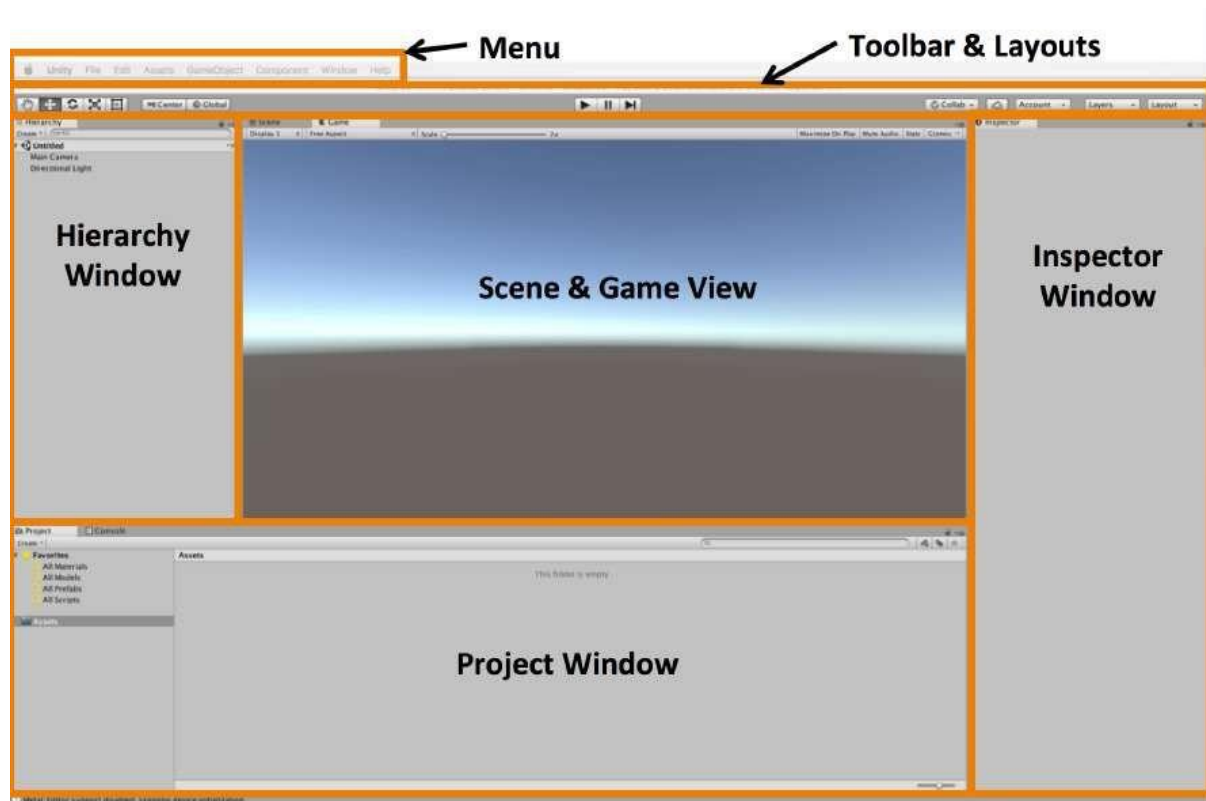
□ Time.timeScale:

1. Time.timeScale is the scale at which time passes.
2. This can be used for slow motion effects or to speed up your application. When timeScale is 1.0, time passes as fast as real time. When timeScale is 0.5 time passes 2x slower than real-time.

```
void Update()
{
    if (Input.GetButtonDown("Fire1"))
    {
        if (Time.timeScale == 1.0f)
            Time.timeScale = 0.7f;
        else
            Time.timeScale = 1.0f;
        // Adjust fixed delta time according to timescale
        // The fixed delta time will now be 0.02 real-time seconds per frame
        Time.fixedDeltaTime = this.fixedDeltaTime * Time.timeScale;
    }
}
```

Unity Interface Overview

The Unity interface consists of several key parts. The **Scene View** allows you to visualize and position your Game Objects, which are the building blocks of your game. The **Game View** lets you preview your game as players will see it. The **Hierarchy** window organizes and manages your Game Objects within the scene. The **Inspector** window lets you edit properties of the selected Game Object and its components. The **Project** window is where you manage all your game assets and scripts, and the **Console** window displays errors, warnings, and other messages.



File

Unity's **File** menu includes access to your game's scenes and projects. We will use these features throughout our game development process. As you can see in the following screenshot, we also have access to the **Build Settings**.



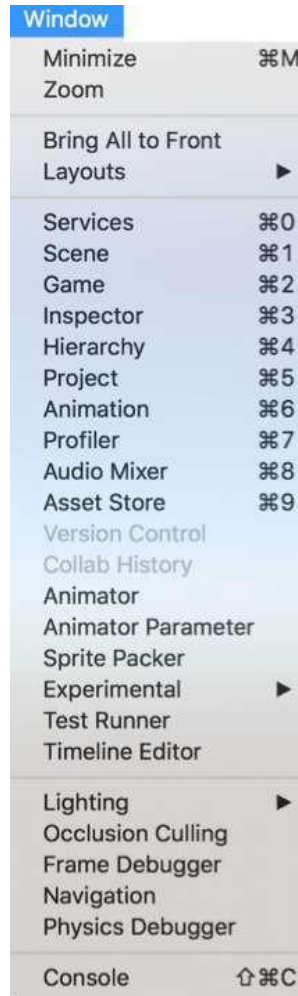
Edit

The **Edit** menu has similar functionality to standard editors, not just game engines. For example, the standard **Cut**, **Copy**, **Paste**, **Delete**, **Undo**, and **Redo** options are there.

Edit	
Undo	⌘Z
Redo	⇧⌘Z
Cut	⌘X
Copy	⌘C
Paste	⌘V
Duplicate	⌘D
Delete	⌘⌫
Frame Selected	F
Lock View to Selected	⇧F
Find	⌘F
Select All	⌘A
Play	⌘P
Pause	⇧⌘P
Step	⌘⇧P
Sign in...	
Sign out	
Selection	▶
Project Settings	▶
Graphics Emulation	▶
Network Emulation	▶
Snap Settings...	

Window

The **Window** menu option provides access to a lot of extra features. As you can see here, there is a **Minimize** option that will minimize the main Unity editor window. The **Zoom** option toggles full screen and zoomed view:



The following table provides a brief description for the remaining options available via the **Window** menu item.

Window Option	Description
Services	Access to integrated services: Ads, Analytics, Cloud Build, Collaborate, Performance Reporting, In-App Purchasing, and Multiplayer.
Scene	Brings focus to the Scene view. Opens the window if not already open.
Game	Brings focus to the Game view. Opens the window if not already open.
Inspector	Brings focus to the Inspector window. Opens the window if not already open.
Hierarchy	Brings focus to the Hierarchy window. Opens the window if not already open.

Project	Brings focus to the Project window. Opens the window if not already open.
Animation	Brings focus to the Animation window. Opens the window if not already open.
Profiler	Brings focus to the Profiler window. Opens the window if not already open.
Audio Mixer	Brings focus to the Audio Mixer window. Opens the window if not already open.
Asset Store	Brings focus to the Asset Store window. Opens the window if not already open.
Version Control	Unity provides functionality for most popular version control systems.
Collab History	If you are using an integrated collaboration tool, you can access the history of changes to your project here.
Animator	Brings focus to the Animator window. Opens the window if not already open.
Animator Parameter	Brings focus to the Animator Parameter window. Opens the window if not already open.
Sprite Packer	Brings focus to the Sprite Packer window. Opens the window if not already open. In order to use this feature, you will need to enable Legacy Sprite Packing in Project Settings .
Experimental	Brings focus to the Experimental window. Opens the window if not already open. By default, the Look Dev experimental feature is available. Additional experimental features can be found in the Unity Asset Store .
Test Runner	Brings focus to the Experimental window. Opens the window if not already open. This is a tool that runs tests on your code both in edit and play modes. Builds can also be tested.
Timeline Editor	Brings focus to the Timeline Editor window. Opens the window if not already open. This is a contextual menu item.

Lighting	Access to the Lighting window and the Light Explorer window.
Occlusion Culling	This feature allows you to select and edit how objects are drawn. With occlusion culling, only the objects within the current camera's visual range, and not obscured by other objects, are rendered.
Frame Debugger	This feature allows you to step through a game, one frame at a time, so you can see the draw calls on a given frame.
Navigation	Unity's navigation system allows us to implement artificial intelligence with regards to non-player character movement.
Physics Debugger	Brings focus to the Physics Debugger window. Opens the window if not already open. Here we can toggle several physics-related components to help debug physics in our games.
Console	Brings focus to the Console window. Opens the window if not already open. The Console window shows warnings and errors. You can also output data here during gameplay, which is a common internal testing approach.

Scene View

The Scene View is your primary workspace where you create and arrange your game 's objects. You can navigate the scene using the mouse and keyboard shortcuts to move, rotate, and zoom in on Game Objects.

- **Navigation:** Use the mouse and keyboard shortcuts to move, rotate, and zoom in on Game Objects.
- **Tools:** Access tools like Move, Rotate, Scale, and Rect Transform from the toolbar.
- **Gizmos:** Toggle visibility of different scene elements to aid in development.

Toolbar

Unity's toolbar, depicted here, is located at the very top of the editor's interface. It spans the entire width of the window, and spacing is dependent on the current width.



The toolbar can be, theoretically, organized into the following categories: **Transform Tools**, **Gizmo Toggles**, Play Buttons, Cloud and **Account** Buttons, and **Layers** and **Layouts**. We covered the Play Buttons in the *Game view* section; the remaining toolbar categories are detailed as follows.

Transform tools

The transform tools are a set of five fundamental tools for manipulating Game Objects in the **Scene** view. Each tool is represented by an icon:



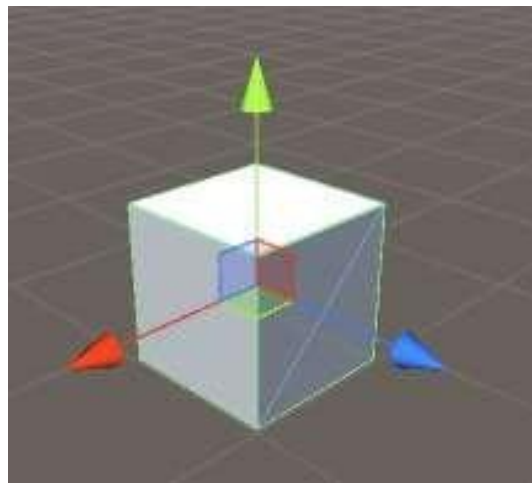
The first button is the Hand tool or View tool. When this tool is selected, our cursor in the **Scene** view turns to a hand. This lets us know which mode we are in. With this tool selected, we can scroll with our mouse to zoom in and out of the scene. If you click on the left mouse button, you are able to pan around the scene. With the right mouse button clicked, you are able to look around, based on the current position of your cursor.

If you hold down the *Alt* key on a PC or *Option* key on a Mac and click on the left mouse button, you can orbit around the current area. Pressing

that same key and the right mouse button allows you to zoom in and out of the scene.

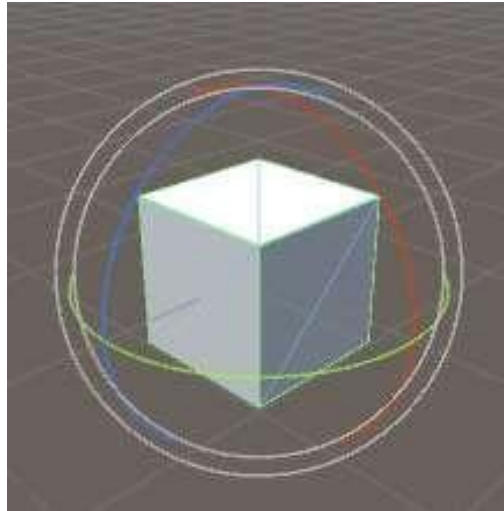
The second button is the Translate tool and is in the shape of a quad arrow. When an object is selected, click on the translate tool; the object will have three gizmos, one for each axis.

Clicking and dragging any of these gizmos moves the object along the respective axes, as shown in the following screenshot:



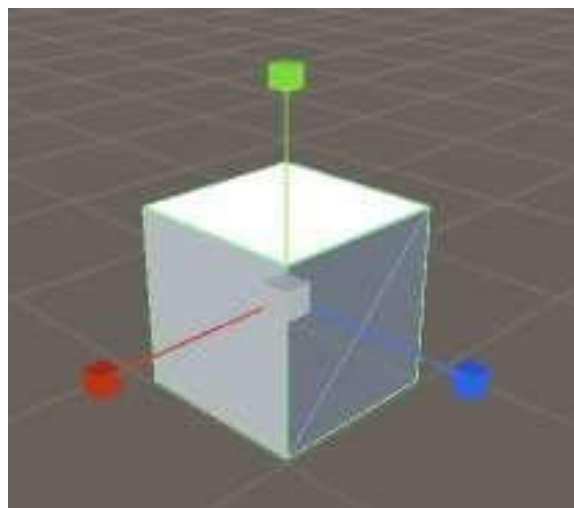
Cube with Transform Translate Tool

The third transform tool is the Rotate tool, which looks like two rotating arrows. This tool allows us to rotate an object along any axis (x, y, or z). Instead of line and arrow gizmos, this tool is instantiated with three colored rings, one for each axis. Clicking a ring and dragging it rotates the object along that axis, as shown in the following screenshot:



Cube with Transform Rotate Tool

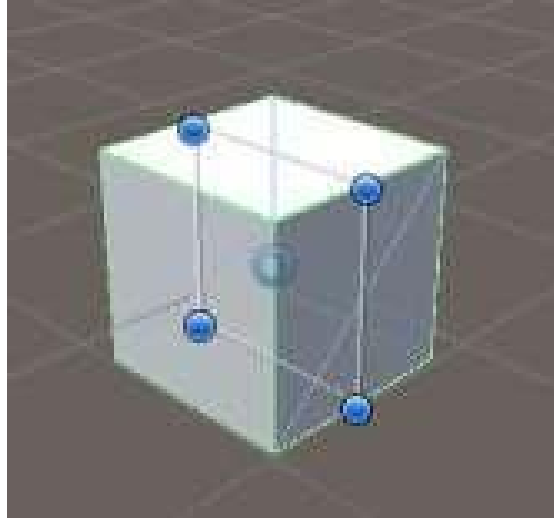
The fourth transform tool is the Scale tool, which is represented with line and block gizmos. Like the other transform tools, there is one gizmo for each axis. Clicking and dragging one of these gizmos increases or decreases the object along the selected axis. For example, you can make a cube wider, narrower, taller, or shorter. If you want to maintain aspect ratio, you can click on the centre square instead of the red, blue, or green square. Now, when you click and drag, your object will grow or shrink in perfect aspect ratio, as shown in the following screenshot:



Cube with Transform Scale Tool

The final transform tool is the Rect tool and is represented by a rectangle with intersecting points. The Rect tool can be used to move, resize, and

rotate an object in the **Scene** view. So, this is a versatile tool that also has corresponding properties that you can edit directly using the **Inspector** view. Take a look at the following screenshot:



Cube with Transform Rect Tool

Game View

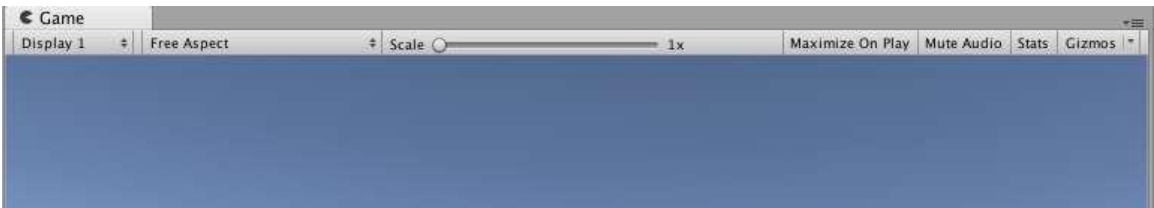
The Game View simulates how your game will look and feel to players. It runs the game in real-time, allowing you to test and play your game within the Unity Editor. You can make changes to your game while you are in **Game** mode. Any changes you make while in **Game** mode are temporary. Once you get out of **Game** mode, those changes will be automatically undone.



- **Play Mode:** Click the Play button to enter Play Mode and test your game.
- **Resolution:** Adjust the resolution to see how your game looks on different screen sizes.
- **Stats:** View real-time statistics like FPS, draw calls, and memory usage.

As shown in the following screenshot, the **Game** view has a set of controls lining the top of the interface. This is referred to as the control

bar:



The following table includes each of the components of the above mentioned control bar. The image, name of control, and description are provided for each component:

Icon	Control	Description
	Display	If you have multiple cameras in your scene, you can change the view to a specific camera.
	Aspect	Available aspect ratios are:
		• Low Resolution
		Aspect Ratios
		• Free Aspect
		• 5:4
		• 4:3
		• 3:2
		• 16:10
	Scale Slider	• 16:9
		• Standalone (1024x768)
		This slider allows you to zoom in to a specific area for a more detailed view.

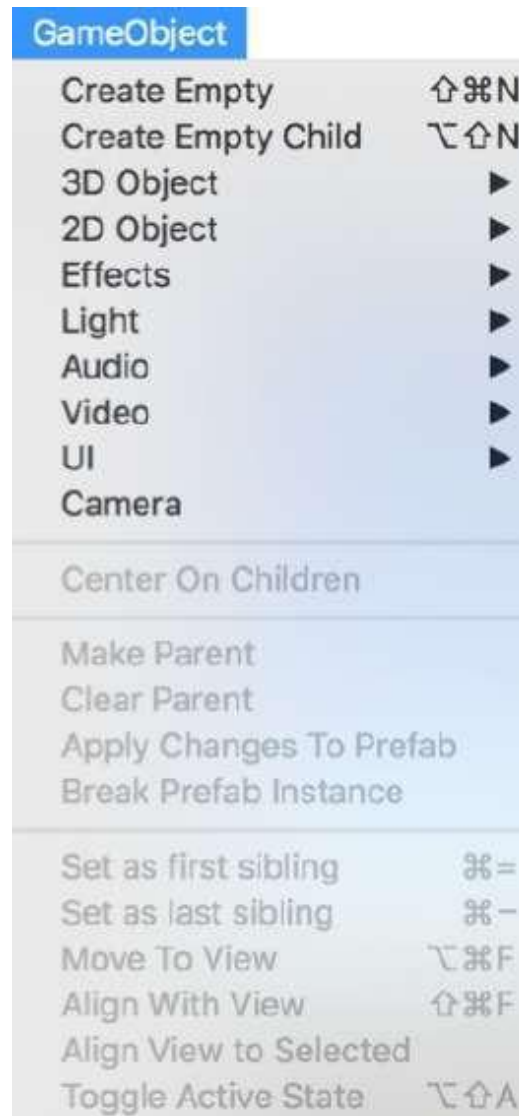
	Maximize On Play	This control is a toggle that lets you maximize the Game view to the full size of your editor when in Game mode.
	Mute Audio	This toggle allows you to mute the game's audio when in Game mode.
	Stats	This control toggles the statistics overlay on/off.
	Gizmos	This powerful search feature enables you to filter objects in the Game view.

Hierarchy Panel

The Hierarchy Window lists all GameObjects in the current scene. You can select, organize, and manage these objects from here. GameObjects can be nested to create parent-child relationships, which helps in organizing complex scenes.

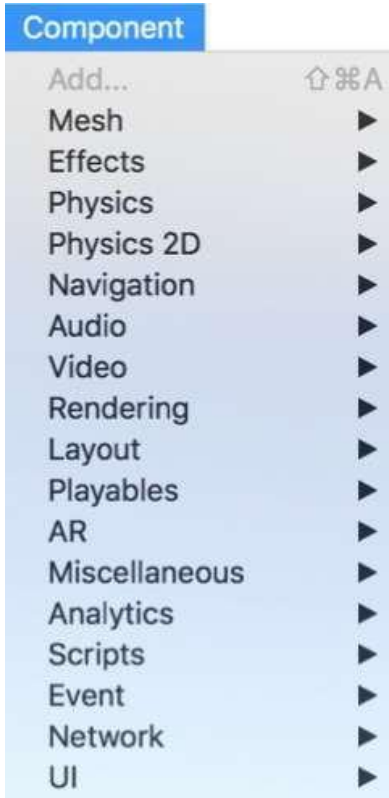
- **Game Objects:** Every entity in the scene, such as characters, lights, cameras, etc. **Parent-Child Relationships:** Organize objects hierarchically to manage complex scenes.
- **Create Menu:** Right-click to quickly add new Game Objects like lights, cameras, or empty objects. Following is the screenshot of the Game objects available in hierarchy section.





Inspector Panel

The Inspector Window displays detailed information about the currently selected Game Object. You can modify the properties of Game Objects and their components here. For example, you can change the position, scale, and rotation of a Game Object, or tweak the settings of a Rigid body component.



- **Components:** Add or remove components to modify Game Object behaviour. In the above screenshot are the components available in Unity.
- **Transform:** Adjust position, rotation, and scale properties.
- **Scripts:** Attach scripts and modify their public variables.
- **Colliders :** A **collider** is a Unity component that defines the shape of a GameObject for the purposes of physical collisions. Colliders are invisible, and do not need to be the same shape as the GameObject's mesh.

Colliders are of two types Collider 2D and Collider 3D applied to 2D and 3D mesh respectively

- **RigidBody:** Rigidbodies enable your GameObjects to act under the control of physics. The Rigidbody can receive forces and torque to make your objects move in a realistic way. Any GameObject must contain a Rigidbody to be influenced by gravity, act under added forces via scripting, or interact with other objects through the NVIDIA PhysX physics engine.

RigidBody are of two types Rigidbody 2D and Rigidbody 3D applied to 2D and 3D mesh respectively

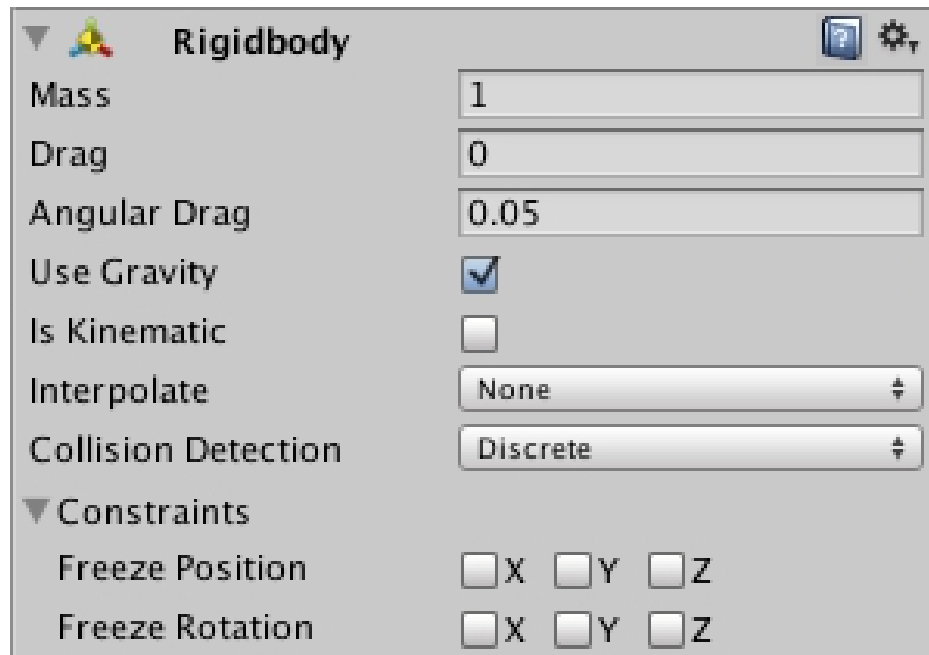
Properties

• Property:	• Function:
• Mass	• The mass of the object (in kilograms by default).
• Drag	• How much air resistance affects the object when moving from forces. 0 means no air resistance, and infinity

• Property:	• Function:
	makes the object stop moving immediately.
• Angular Drag	How much air resistance affects the object when rotating from torque. 0 means no air resistance. Note that you cannot make the object stop rotating just by setting its Angular Drag to infinity.
• Use Gravity	If enabled, the object is affected by gravity.
• Is Kinematic	If enabled, the object will not be driven by the physics engine, and can only be manipulated by its Transform. This is useful for moving platforms or if you want to animate a Rigidbody that has a HingeJoint attached.
• Interpolate	Try one of the options only if you are seeing jerkiness in your Rigidbody's movement.
• - None	No Interpolation is applied.
• - Interpolate	Transform is smoothed based on the Transform of the previous frame.

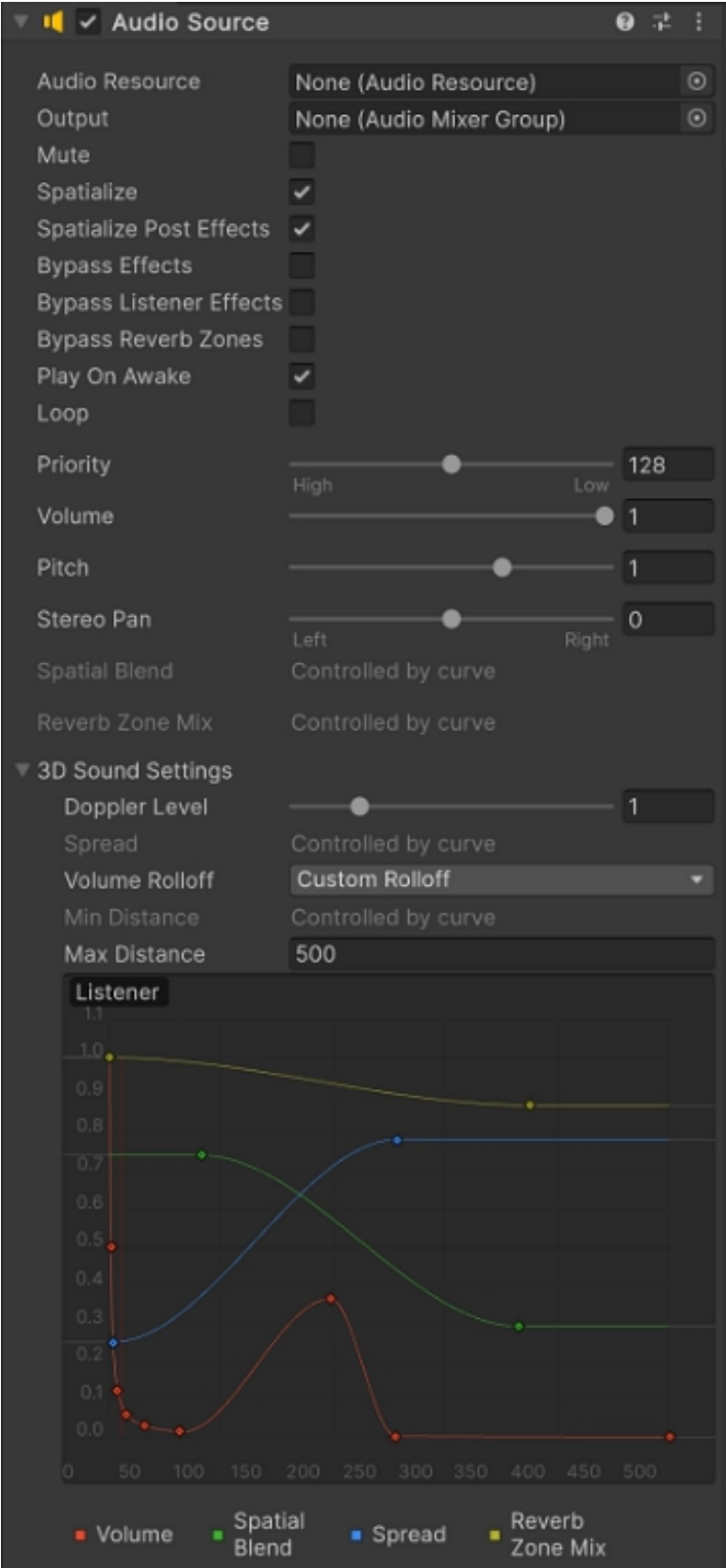
• Property:	• Function:
• - Extrapolate	• Transform is smoothed based on the estimated Transform of the next frame.
• Collision Detection	• Used to prevent fast moving objects from passing through other objects without detecting collisions.
• Discrete	• Use Discrete collision detection against all other colliders in the scene. Other colliders will use Discrete collision detection when testing for collision against it. Used for normal collisions (This is the default value).
• Continuous	• Use Discrete collision detection against dynamic colliders (with a rigidbody) and continuous collision detection against static MeshColliders (without a rigidbody). Rigidbodies set to Continuous Dynamic will use continuous collision detection when testing for collision against this rigidbody. Other rigidbodies will use Discrete Collision detection. Used for objects which the Continuous Dynamic detection needs to collide with. (This has a big impact on physics performance, leave it set to Discrete, if you don't have issues with collisions of fast objects)

• Property:	• Function:
• Continuous Dynamic	• Use continuous collision detection against objects set to Continuous and Continuous Dynamic Collision. It will also use continuous collision detection against static MeshColliders (without a rigidbody). For all other colliders it uses discrete collision detection. Used for fast moving objects.
• Constraints	• Restrictions on the Rigidbody's motion:-
• Freeze Position	• Stops the Rigidbody moving in the world X, Y and Z axes selectively.
• Freeze Rotation	• Stops the Rigidbody rotating around the local X, Y and Z axes selectively.



- **Audio Source:** The Audio Source plays back an Audio Clip in the scene
. The clip can be played to an audio listener or through an audio mixer. The audio source can play any type of Audio Clips and can be configured to play these as 2D, 3D, or as a mixture (*SpatialBlend*). The audio can be spread out between speakers (stereo to 7.1) (*Spread*) and morphed between 3D and 2D (*SpatialBlend*). This can be controlled over distance with falloff curves. Also, if the listener is within one or multiple Zones, reverb is applied to the source. Individual filters can be applied to each audio source for an even richer audio experience.

You can play a single audio clip using Play, Pause and Stop. You can also adjust its volume while playing using the volume property, or seek using time. Multiple sounds can be played on one AudioSource using PlayOneShot. You can play a clip at a static position in 3D space using PlayClipAtPoint.



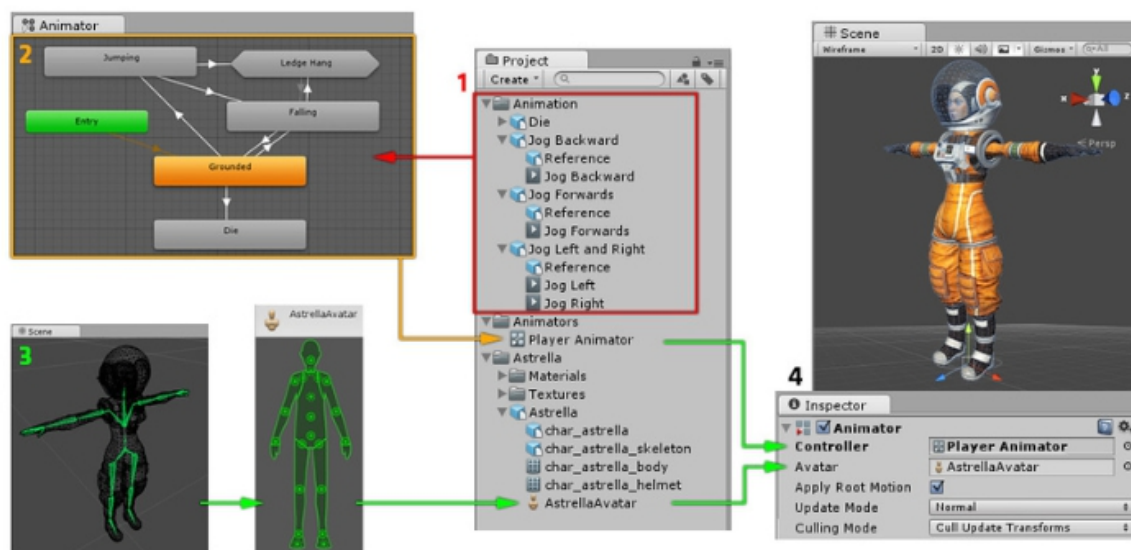
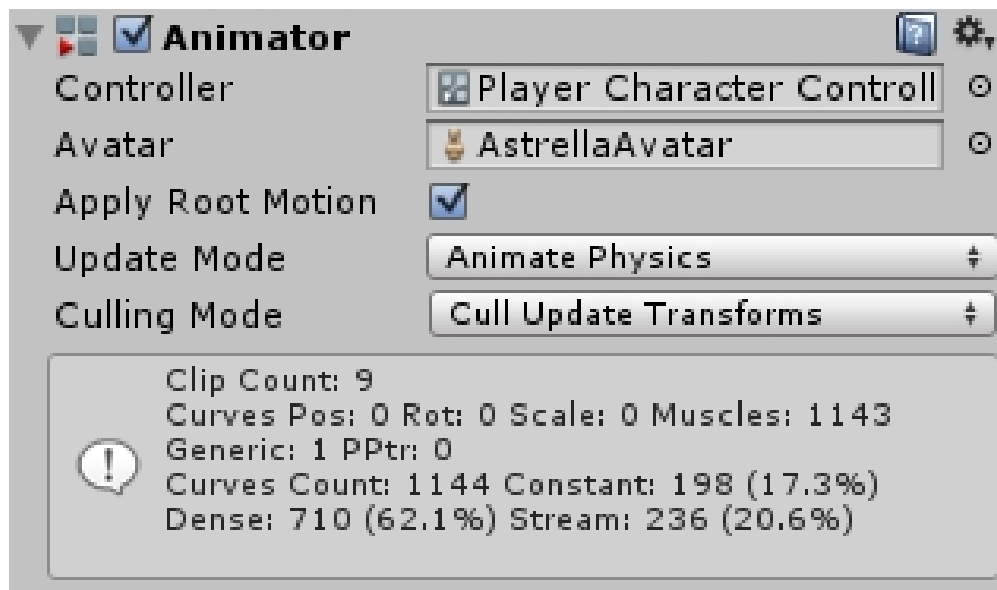
Properties

Property	Description
Audio Clip	Reference to the sound clip file that will be played.
Output	By default, the clip is output directly to the Audio Listener in the Scene. Use this property to output the clip to an Audio Mixer instead.
Mute	If enabled the sound will be playing but muted.
Spatialize	Enables or disables custom spatialization for the Audio Source. This property is only available if you have installed an audio spatializer SDK , and selected it in your project's global audio settings.
Spatialize Post Effect	Determines whether the custom spatializer is applied before or after other effects attached to the Audio Source. Enable this property to apply the custom spatialize after other Effects attached to the Audio Source. This property is only available if you have enabled the Spatialize property for the Audio Source.
Bypass Effects	This is to quickly “by-pass” filter effects applied to the audio source. pan easy way to turn all effects on/off.
Bypass Listener Effects	This is to quickly turn all Listener effects on/off.
Bypass Reverb Zones	This is to quickly turn all Reverb Zones on/off.

Play On Awake	If enabled, the sound will start playing the moment the scene launches. If disabled, you need to start it using the Play() command from scripting.
Loop	Enable this to make the Audio Clip loop when it reaches the end.
Priority	Determines the priority of this audio source among all the ones that coexist in the scene. (Priority: 0 = most important. 256 = least important. Default = 128.). Use 0 for music tracks to avoid it getting occasionally swapped out.
Volume	How loud the sound is at a distance of one world unit (one meter) from then Audio Listener
Pitch	Amount of change in pitch due to slowdown/speed up of the Audio Clip . Value 1 is normal playback speed.
Stereo Pan	Sets the position in the stereo field of 2D sounds.
Spatial Blend	Sets how much the 3D engine has an effect on the audio source.
Reverb Zone Mix	Sets the amount of the output signal that gets routed to the reverb zones. The amount is linear in the (0 - 1) range, but allows for a 10 dB amplification in the (1 - 1.1) range which can be useful to achieve the effect of near-field and distant sounds.
3D Sound Settings	Settings that are applied proportionally to the Spatial Blend parameter.

Doppler Level	Determines how much doppler effect will be applied to this audio source (if is set to 0, then no effect is applied).
Spread	Sets the spread angle to 3D stereo or multichannel sound in speaker space.
Min Distance	Within the MinDistance, the sound will stay at loudest possible. Outside MinDistance it will begin to attenuate. Increase the MinDistance of a sound to make it 'louder' in a 3d world, and decrease it to make it 'quieter' in a 3d world.
Max Distance	The distance where the sound stops attenuating at. Beyond this point it will stay at the volume it would be at Max Distance units from the listener and will not attenuate any more.
Rolloff Mode	How fast the sound fades. The higher the value, the closer the Listener has to be before hearing the sound.
Logarithmic Rolloff	The sound is loud when you are close to the audio source, but when you get away from the object it decreases significantly fast.
Linear Rolloff	The further away from the audio source you go, the less you can hear it.
Custom Rolloff	The sound from the audio source behaves accordingly to how you set the graph of roll offs.

- **Animator:** Use the Animator component to assign animation to a GameObject in your Scene.
The Animator component requires a reference to an Animator Controller which defines which animation clips to use, and controls when and how to blend and transition between them.



Properties

Property:	Function:
Controller	The animator controller attached to this character.
Avatar	The Avtar for this character. (If the Animator is being used to animate a humanoid character)

Property:	Function:
Apply Root Motion	Select whether to control the character's position and rotation from the animation itself or from script.
Update Mode	This allows you to select when the Animator updates, and which timescale it should use.
Normal	The Animator is updated in-sync with the Update call, and the animator's speed matches the current timescale. If the timescale is slowed, animations will slow down to match.
Animate Physics	The animator is updated in-sync with the FixedUpdate call (i.e. in lock-step with the physics system). You should use this mode if you are animating the motion of objects with physics interactions, such as characters which can push rigidbody objects around.
Unscaled Time	The animator is updated in-sync with the Update call, but the animator's speed ignores the current timescale and animates at 100% speed regardless. This is useful for animating a GUI system at normal speed while using modified timescales for special effects or to pause gameplay.
Culling Mode	Culling mode you can choose for animations.
Always Animate	Always animate, don't do culling even when offscreen.
Cull Update Transforms	Retarget, IK and write of Transforms are disabled when renderers are not visible.
Cull Completely	Animation is completely disabled when renderers are not visible.

Layers and Layouts

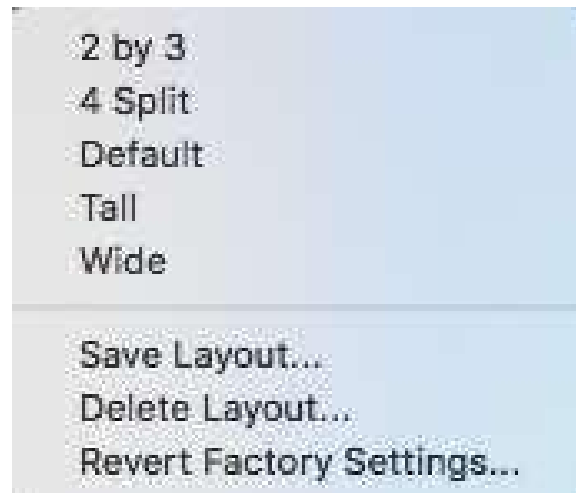
The final section of buttons on the toolbar consists of the **Layers** drop-down and the **Layout** drop-down buttons. These buttons are illustrated beneath and are located in the far-right end of the toolbar:



Selecting the **Layers** drop-down button allows you to select what layers you want to view, not view, and which ones you want to lock. Also, as you can see here, you have access to edit layers:



Editing layers consists of creating user-defined layers that you can reorder and rename. The **Layout** drop-down reveals the set of options:



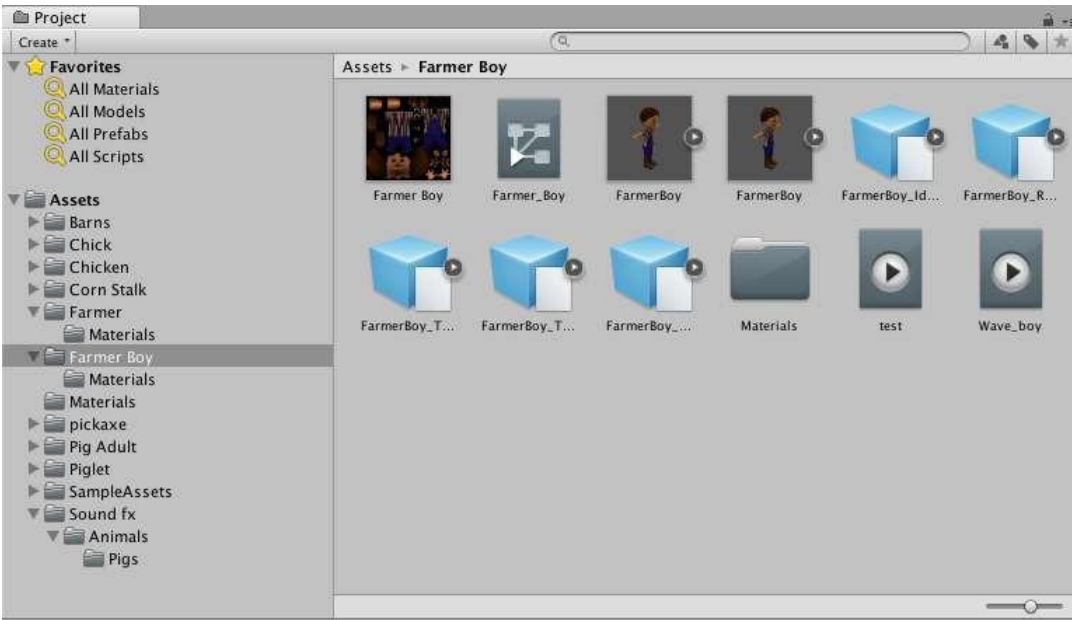
Layouts

One of the wonderful things about working with Unity is that you can customize the way the user interface is laid out. You can use one of the predefined layouts of **2 by 3**, **4 Split**, **Tall**, or **Wide**, or you can create your own. Layouts refer to how the various views in Unity are arranged on the screen.

To change a layout, we simply click on the **Layout** button that is located in the far top-right corner of the Unity interface.

Project Panel

The Project Window shows all the assets in your project. Assets include everything from scripts and textures to models and sounds. You can organize assets into folders to keep your project structured.



- **Asset Management:** Store and manage all your game assets, including scripts, textures, models, sounds, and prefabs.
 - **Scripts:** Store and manage your C# scripts here.
 - **Textures:** Store image files like PNGs and JPEGs.
 - **Models:** Import 3D models in formats like FBX.
 - **Sounds:** Manage audio files such as MP3 and WAV.
 - **Prefabs:** Reusable Game Objects saved as prefabs for easy instantiation.
 - **Materials:** Define how surfaces are rendered with textures and shaders.
- **Organization:** Create folders to organize your assets logically.
- **Importing:** Right-click to import new assets into your project.

Console Window

The Console Window displays messages from Unity, including errors, warnings, and logs. This is an essential tool for debugging and understanding what 's happening in your game during development.

- **Logs:** View debug logs, warnings, and errors.
- **Search:** Filter messages to find specific logs or errors.
- **Clear:** Clear the console to remove old messages and focus on new ones.

Key Components

In Unity, **GameObjects** are the fundamental objects that represent everything in your game, such as characters, props, and scenery.

Components are added to GameObjects to give them functionality, such as a Rigidbody for physics or a Collider for collision detection. **Scenes** are individual levels or screens in your game, where you arrange and manage your GameObjects.

Chapter 4: Creating Your First Project

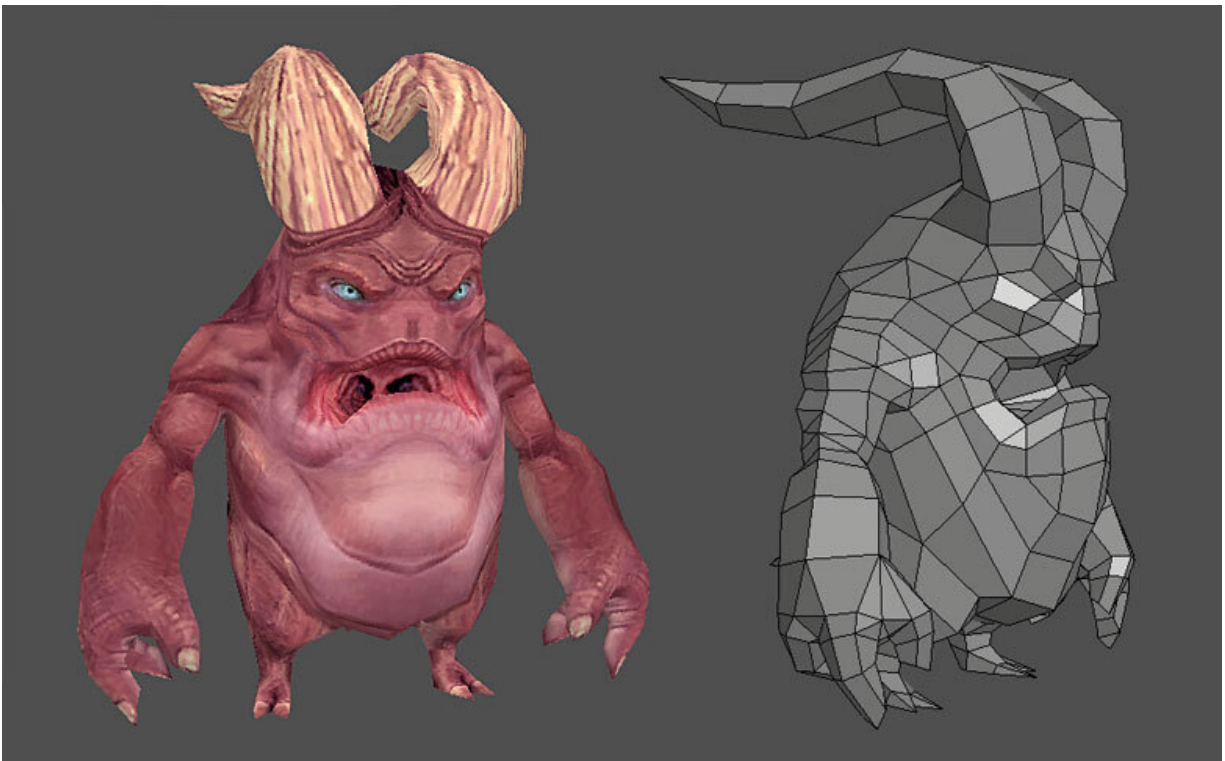
Importing Asset From 3D Software To Unity Engine

Working with gaming pipeline, Can also be called as rules of making models for game engine.

Creating An Object in 3D Software:

- Open your 3D software like Maya, Blender or 3DsMax.
- Create a model in Low poly and to smooth it, smooth the edges.
- Do not use smooth modifiers on any object as it increases the polycount hence, increasing the model size.
- Keep the mesh flow clean and reduce the poly count manually.
- Keep the mesh optimised and low-poly and use textures to give details in the model.
- Unwrap the model properly using UVW unwrap to create texture in a proper flow, unwrap the model in a way that there should be only one material for the texture and one UV map of that model.
- However if there is requirement of secondary material then there can be two or more materials.

- Create a collage of UV map as it will result in a collage of textures.
- Do not use auto unwrap on a model, as it creates individual UV ' s of each part of the model and that creates baking issues in Unity.
- Use Software ' s like Substance painter, Mari, etc to created high quality textures.
- Use texture software ' s to give details to the model and Generate AO map, Normal Map, Base map/Albedo map other maps like Height map, Roughness map, etc are optional and depends on the model.
- Back to the 3D Software, Freeze transform the model and export it in .fbx and disable embedded media as it will export the model with textures .



Following is an example of Low poly mesh with details in textures.

Editing Textures in Photoshop:

- Import all the generated textures in Photo Shop and bake the AO map on the Albedo map.
- This will give shadow effect of the AO map to the Albedo map itself.
- Export the Normal map, Albedo map and other texture maps from photo shop.
- Use Shortcut key Alt+Ctrl+Shift+S to export for web and choose Jpeg and give dimension of the image to be 512X512.
- This will reduce the texture size keeping the quality of textures.
- If there is a glass texture then export it as PNG.
- If the glass texture is in same collage then cut that particular part and keep it empty to create an illusion of transparent glass.
- If it ' s a tainted glass then instead of that empty place a tainted glass texture can be added.

Setting Up a New Project

To create a new project in Unity, open Unity Hub and click the "New" button. Select a template based on the type of game you want to create, such as 2D or 3D. Enter a name and location for your project, then click "Create" to open your new project in Unity Editor.

Understanding the Project Structure

Your Unity project will have a specific structure. The **Assets** folder contains all your game assets, such as scripts, textures, and models. **Scenes** are stored as individual files within this folder, representing different levels or screens in your game. **Scripts** contain the C# code that controls your game logic and behaviour.

Setting up The Arena:

- To import the model in Unity right click in assets section under project panel and select ' **Folder** ' and rename it.

- Open the folder and right click again then select ‘ **Import new asset** ’ .
- Select the fbx file and import it .
- Once imported select the fbx file and at the right side in Inspector panel there will open all the properties of this fbx file.
- Select materials and export it , as the embedded media was off while exporting the file , the ‘ **Export Texture** ’ option will be disabled.
- Once the materials are exported go to ‘ **Animations** ’ section beside ‘ **Materials** ’ and deselect ‘ **Import Animations** ’ if there are no animations.
- Goto ‘ Models ’ section and untick ‘ **Import cameras** ’ and ‘ **Import Lights** ’ and Tick ‘ **Generate Lightmap UVs** ’ and click Apply.
- Once done with above steps create two new folders named **Textures** and **Materials**.
- Create a material in materials folder by right clicking<Create<Materials and rename it same as the model name.
- Import the textures in textures folder and then select the material.
- Assign textures on the material according to the name .
- Once done with this assign the material to the model.
- Right click in the assets folder<Create<Prefab.
- Rename the prefab same as the model and add prefab at the end.
- Drag and drop the fbx model onto the prefab and drag and drop the prefab to the Hierarchy panel .
- The model will appear at exact position as required and will have all the textures .

Chapter 5: Lights, Camera, Shadows

Working with cameras

Cameras render scenes so that the user can view them. Think about the hidden complexity in that statement. Our games are 3D, but people playing our games view them on 2D displays such as a televisions, computer monitors, or mobile devices. Fortunately for us, Unity makes implementing cameras easy work.

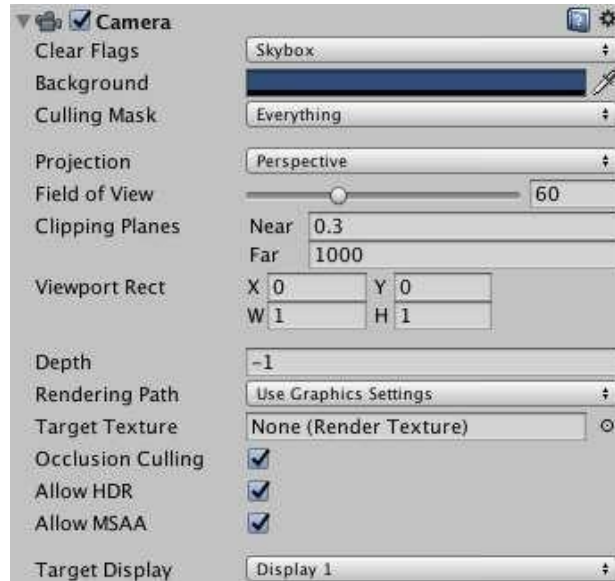
Cameras are Game Objects and can be edited using transform tools in the **Scene** view as well as in the **Inspector** panel. Every scene must have at least one camera. In fact, when a new scene is created, Unity creates a camera named **Main Camera**. As you will see later in this chapter, a scene can have multiple cameras.

In the **Scene** view, cameras are indicated with a white camera silhouette, as shown in the following screenshot:



When we click our **Main Camera** in the **Hierarchy** panel, we are provided with a **Camera Preview** in the **Scene** view. This gives us a preview of what the camera sees as if it were in game mode. We also

have access to several parameters via the **Inspector** panel. The **Camera** component in the **Inspector** panel is shown here:



Let's look at each of these parameters :

- The **Clear Flags** parameter lets you switch between **Skybox**, **Solid Color**, **Depth Only**, and **Don't Clear**. The selection here informs Unity which parts of the screen to clear. We will leave this setting as **Skybox**. You will learn more about Skyboxes later in this chapter.
- The **Background** parameter is used to set the default background fill (color) of your game world. This will only be visible after all game objects have been rendered and if there is no Skybox.
- The **Culling Mask** parameter allows you to select and deselect the layers you want the camera to render. The default selection options are **Nothing**, **Everything**, **Default**, **Transparent FX**, **Ignore Raycast**, **Water**, and **UI**. For our game, we will select **Everything**. If you are not sure which layer a game object is associated with, select it and look at the **Layer** parameter in the top section of the Inspector panel. There you will see the assigned

layer. You can easily change the layer as well as create your own unique layers. This gives you finite rendering control.

- The **Projection** parameter allows you to select which **projection**, **perspective** or **orthographic**, you want for your camera. We will cover both of those projections later in this chapter. When *perspective* projection is selected, we are given access to the **Field of View** parameter. This is for the width of the camera's angle of view. The value range is 1-179°. You can use the slider to change the values and see the results in the **Camera Preview** window. When **orthographic** projection is selected, an additional **Size** parameter is available. This refers to the viewport size. For our game, we will select perspective projection with the **Field of View** set to 60.
- The **Clipping Planes** parameters include **Near** and **Far**. These settings set the closest and furthest points, relative to the camera, that rendering will happen at. For now, we will leave the default settings of 0.3 and 1000 for the **Near** and **Far** parameters, respectively.
- The **Viewport Rect** parameter has four components – **X**, **Y**, **W**, and **H** – that determine where the camera will be drawn on the screen. As you would expect, the **X** and **Y** components refer to horizontal and vertical positions, and the **W** and **H** components refer to width and height. You can experiment with these values and see the changes in the **Camera Preview**. For our game, we will leave the default settings.
- The **Depth** parameter is used when we implement more than one camera. We can set a value here to determine the camera's priority in relation to others. Larger values indicate a higher priority. The default setting is sufficient for our game.
- The **Rendering Path** parameter defines what rendering methods our camera will use. The options are **Use Graphics Settings**,

Forward, Deferred, Legacy Vertex Lit, and Legacy Deferred (light prepass). We will use the **Use Graphics Settings** option for our game, which also uses the default setting.

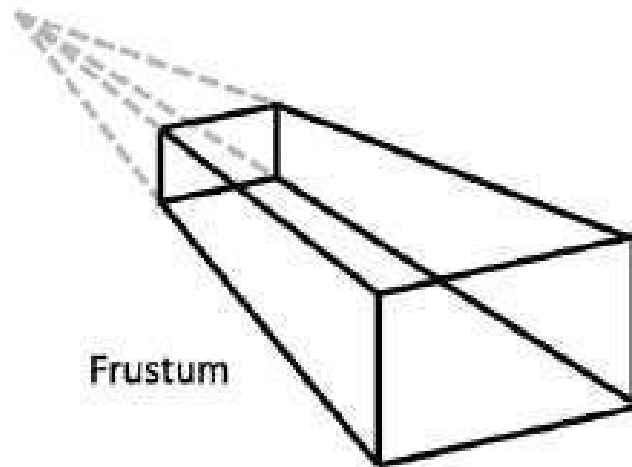
- The **Target Texture** parameter is not something we will use in our game. When a render texture is set, the camera is not able to render to the screen.
 - The **Occlusion Culling** parameter is a powerful setting. If enabled, Unity will not render objects that are occluded, or not seen by the camera. An example would be objects inside a building. If the camera can currently only see the external walls of the building, then none of the objects inside those walls can be seen. So, it makes sense to not render those. We only want to render what is absolutely necessary to help ensure our game has smooth gameplay and no lag. We will leave this as enabled for our game.
- The **Allow HDR** parameter is a checkbox that toggles a camera's **High Dynamic Range (HDR)** rendering. We will leave the default

setting of enabled for our game.

- The **Allow MSAA** parameter is a toggle that determines whether our camera will use a **Multisample Anti-Aliasing (MSAA)** render target. MSAA is a computer graphics optimization technique and we want this enabled for our game.

Orientating your frustum

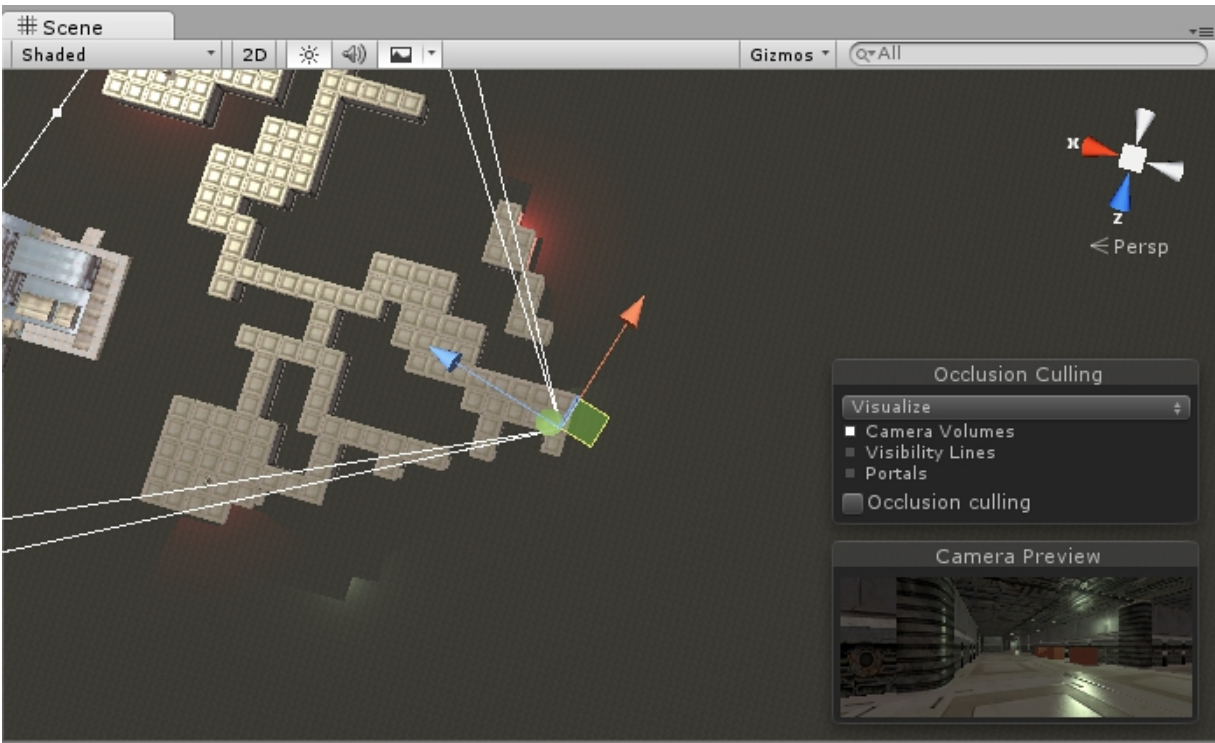
When a camera is selected in the **Hierarchy** view, its frustum is visible in the **Scene** view. A frustum is a geometric shape that looks like a pyramid that has had its top cut off, as illustrated here:



The near, or top, plane is parallel to its base. The base is also referred to as the far plane. The frustum's shape represents the viable region of your game. Only objects in that region are rendered. Using the camera object in **Scene** view, we can change our camera's frustum shape.

- **Frustum Culling**

Frustum Culling only disables the renderers for objects that are outside the camera's viewing area but does not disable anything hidden from view by overdraw. Note that when you use Occlusion Culling you will still benefit from Frustum Culling.

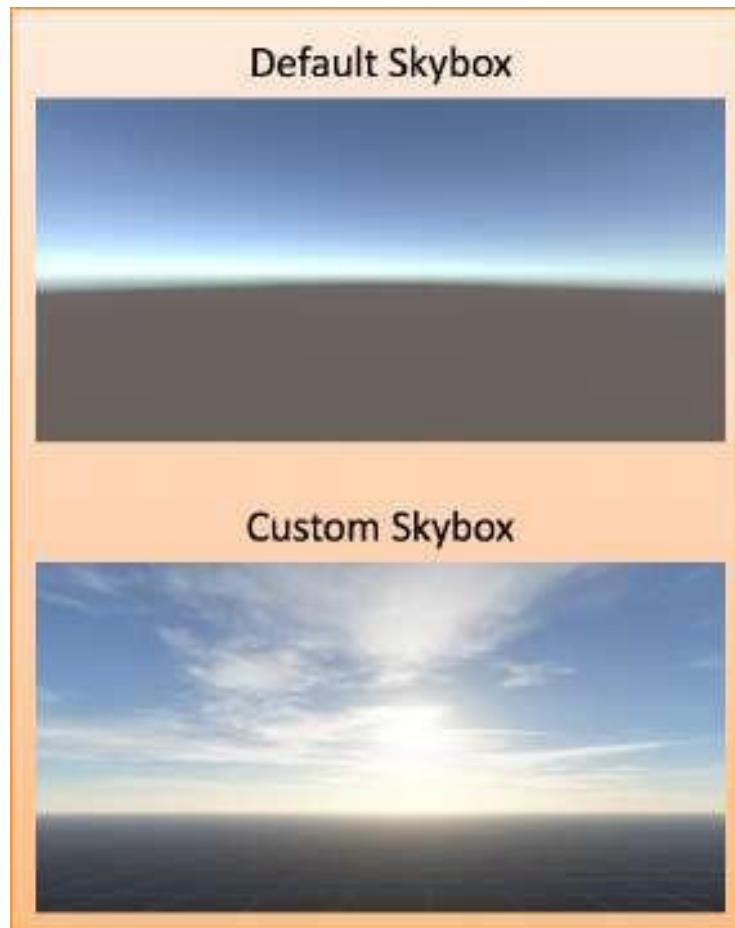


Creating a Skybox

When we create game worlds, we typically create the ground, buildings, characters, trees, and other game objects. What about the sky? By default, there will be a textured blue sky in your Unity game projects. That sky is sufficient for testing but does not add to an immersive gaming experience. We want a bit more realism, such as clouds, and that can be accomplished by creating a Skybox.

A Skybox is a six-sided cube visible to the player beyond all other objects. So, when a player looks beyond your objects, what they see is your Skybox. As we said, Skyboxes are six-sided cubes, which means you will need six separate images that can essentially be clamped to each other to form the cube.

The following screenshot shows the **Default Skybox** that Unity projects start with as well as the completed **Custom Skybox** you will create in this section:

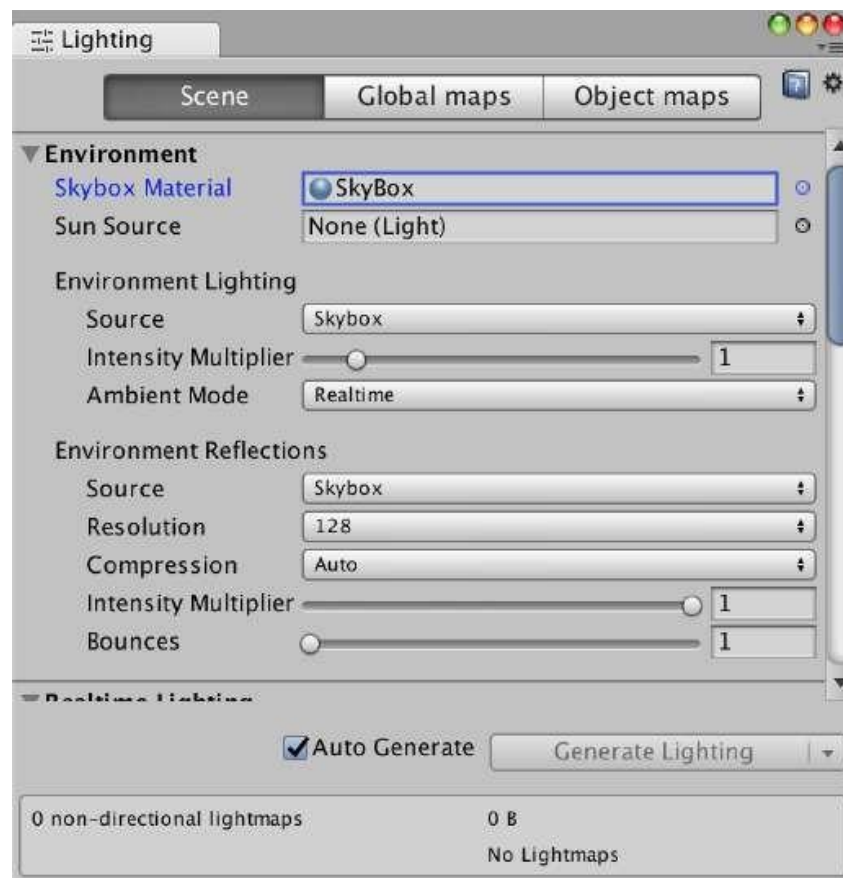


Perform the following steps to create a Skybox:

- In the **Project** panel, create a Skybox subfolder in the Assets folder. We will use this folder to store our textures and materials for the Skybox.
- Drag the provided six Skybox images, or your own, into the new Skybox folder.
- Ensure the Skybox folder is selected in the Project panel.
- From the top menu, select **Assets | Create | Material**. In the **Project** panel, name the material Skybox.
- With the **Skybox** material selected, turn your attention to the **Inspector** panel.
- Select the **Shader** drop-down menu and select **SkyBox | 6 Sided**.
- Use the **Select** button for each of the six images and navigate to the images you added in step 2. Be sure to match the appropriate

texture to the appropriate cube face. For example, the Skybox Front texture matches the Front[+Z] cube face on the **Skybox Material**.

- In order to assign our new Skybox to our scene, select **Window | Lighting | Settings** from the top menu. This will bring up the **Lighting** settings dialog window.
- In the **Lighting** settings dialog window, click on the small circle to the right of the **Skybox Material** input field. Then, close the selection window and the **Lighting** window. Refer to the following screenshot:



You will now be able to see your Skybox in the **Scene** view. When you click on the **Camera** in the **Hierarchy** panel, you will also see the Skybox as it will appear from the camera's perspective.

Working with lighting

Just like in the real world, cameras need lights to show us objects. In Unity games, we use multiple lights to illuminate the game environment.

In Unity, we have both dynamic lighting techniques as well as light baking options for better performance. We can add numerous light sources throughout our scenes and selectively enable or disable an object's ability to cast or receive shadows. This level of specificity gives us tremendous opportunity to create realistic game scenes.

Perhaps the secret behind Unity's ability to so realistically render light and shadows is that Unity models the actual behavior of lights and shadows. Real-time global illumination gives us the ability to instantiate multiple lights in each scene, each with the ability to directly or indirectly impact objects in the scene that are within range of the light sources.

We can also add and manipulate ambient light in our game scenes. This is often done with Skyboxes, a tri-colored gradient, or even a single color. Each new scene in Unity has default ambient lighting, which we can control by editing the values in the **Lighting** window. In that window, you have access to the following settings:

- Environment
 - **Real-time Lighting**
 - **Mixed Lighting**
 - **Lightmapping Settings**
 - **Other Settings**
 - **Debug Settings**

No changes to these are required for our game at this time. We have already set the environmental lighting to our Skybox. In [Chapter 12, Adding Audio and Visual Effects to Our Game](#), we will look at **Fog**, which is available under the **Other Settings** section of the **Lighting** window.

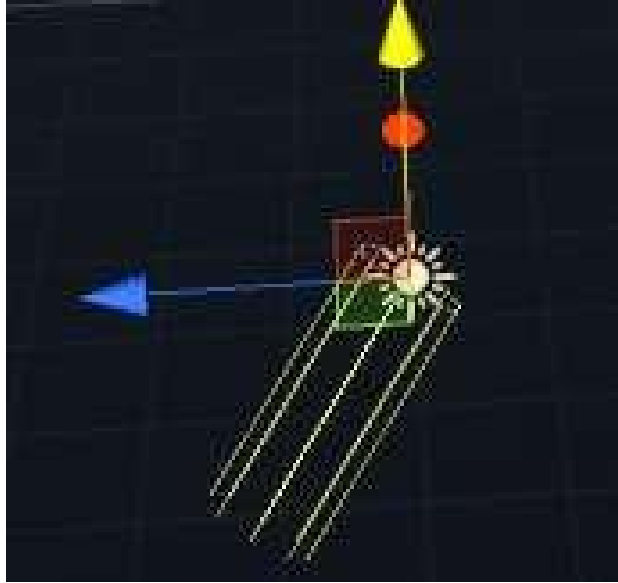
When we create our scenes in Unity, we have three options for lighting. We can use real-time dynamic light, use the baked lighting approach, or use a mixture of the two. Our games perform more efficiently with baked lighting, compared to real-time dynamic lighting, so if performance is a concern, try using baked lighting where you can.

In addition to ambient lighting, there are four types of light: directional, point, spot, and area. We will look at each of these in the following sections.

- **Directional lighting**

When we create a new scene in Unity, a directional light is automatically created for us. This emphasizes the importance of directional lights. This type of light provides illumination in a specific direction. Using transform tools, we have full control over the direction of these lights.

An example of directional lighting is generating sunlight in our scenes. Although the light from directional lights is similar to that of the sun, there is no actual sun-like object that the light comes from. As shown in the following screenshot, **Directional Lights**, when selected, indicate the direction of its illumination with yellow rays:



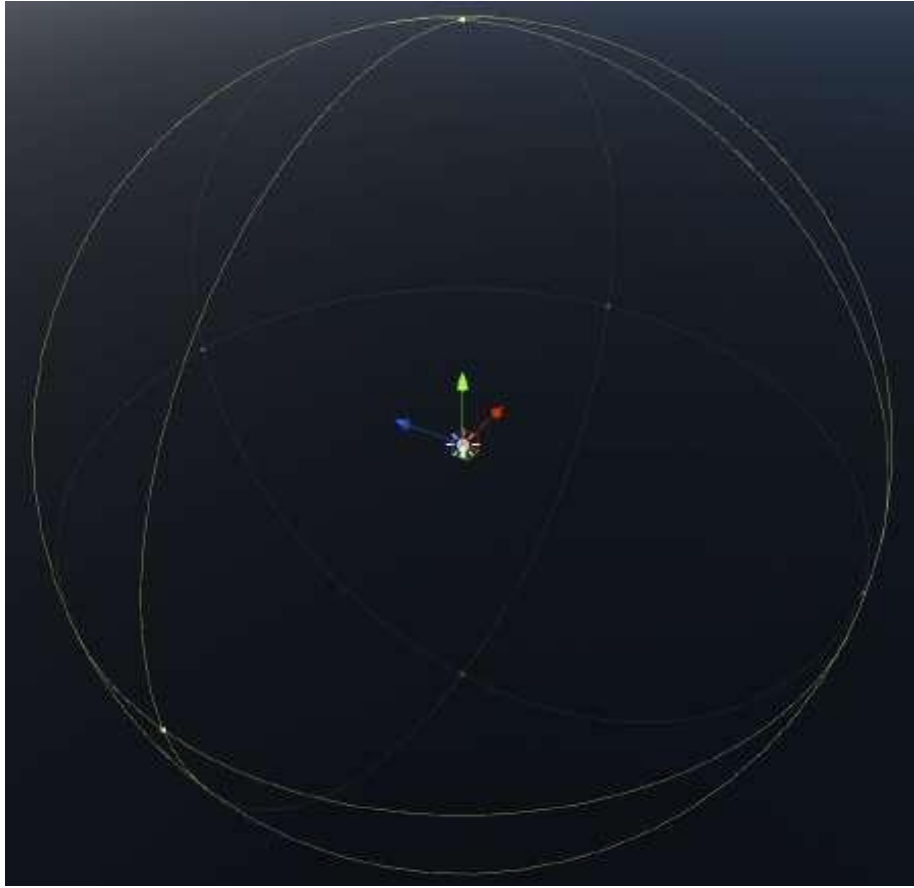
One of the great aspects of using directional lighting is that illumination strength is not dependent on an object's distance from the **Directional Light** object. This type of lighting defines the direction of the light, and distance has no impact on the illumination.

With a **Directional Light** selected, you have access to several parameters in the **Inspector** panel. In addition to the **Transform** section, there is a **Light** section where you can change several settings to include the light color and intensity. The remaining parameters are worth exploring and no changes are required to complete our game.

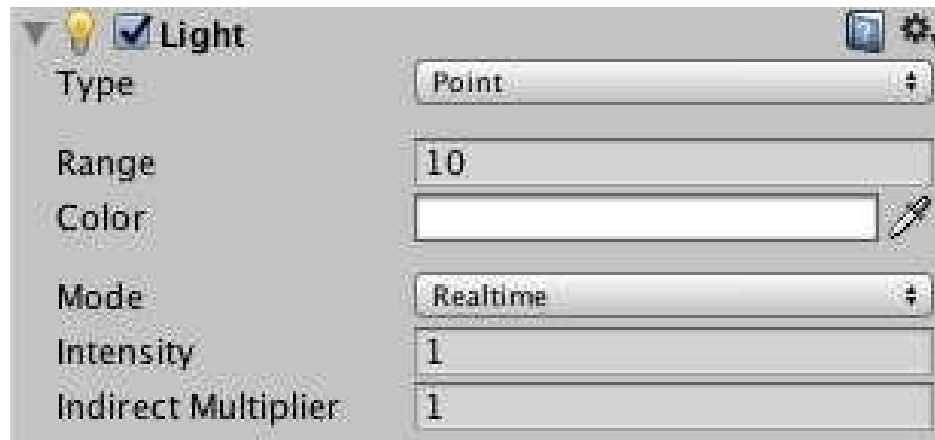
To add additional directional lights, you would select, from the top menu, **GameObject | Light | Directional Light**. You can use the transform tools to modify the location, rotation, and light direction of the light.

- **Point lighting**

Point lights get their name from the fact that they are lights sources emanating from a specific point. These light objects, as indicated in the following diagram, emit light in all directions:



These lights are typically used to simulate fireballs or light bulbs. They can also be used to simulate some magic or special lighting effects. As you can see in the following screenshot, point lights have several properties that affect how they impact the game environment:

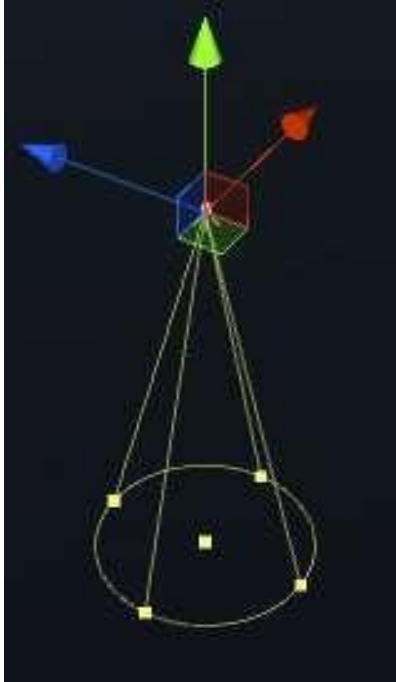


The **Range** is the distance between the center of the light to the outside arc of the light. We can also change the **Color** and **Intensity** to produce the results we desire.

To create a point light, we select **GameObject | Light | Directional Light** from the top menu.

- **Spot lighting**

Spot lights are another type of lighting in Unity. They are meant to provide lighting on a specific spots. Common examples are flashlights and vehicle headlights. As you can see from the following screenshot, the light originates from the source in an outward cone shape:

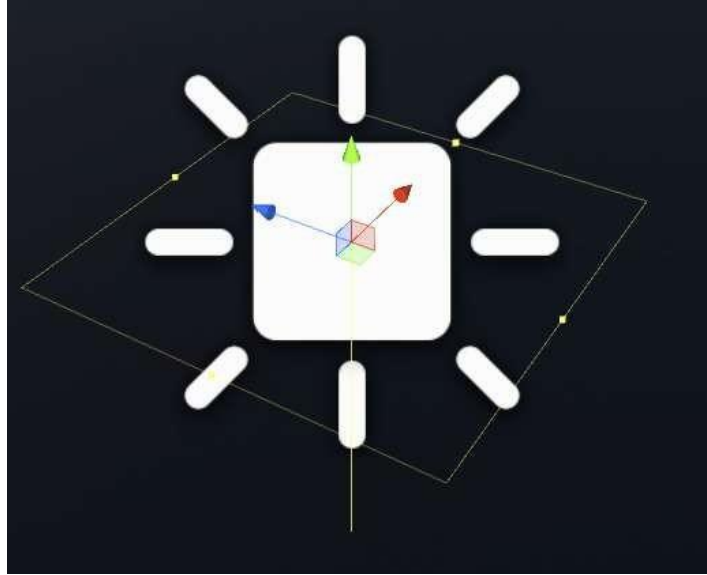


In the **Inspector** panel, we can change the **Range**, **Spot Angle**, **Color** and **Intensity**. **Range**, in this context, refers to the distance between the light source and the furthest point in the cone. The **Spot Angle** is the angle of the outward exterior edge of the cone shape. The **Spot Angle** range is 1-179°. The larger the value is, the larger the cone face will be.

To create a spotlight, we select **Game Object | Light | Spot Light** from the top menu.

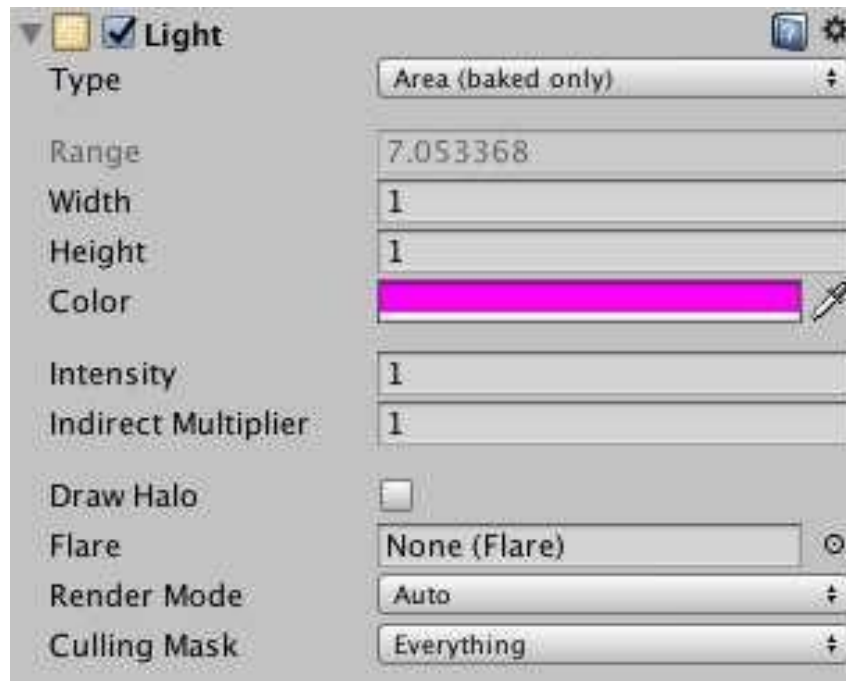
- **Area lighting**

To use an **area light**, we define a rectangle using the transform tools or the **Inspector** panel. Area lights emit light from one side of their rectangle. The following screenshot shows what an area light object looks like in the Unity editor:



Area lights are unique from the other types of light as they can only be baked. This means that real-time rendering will not take place during gameplay. The reason for this is to conduct all the processing regarding area lights prior to gameplay. This processing, if accomplished in real time in a game, would likely result in sufficient lag.

As you can see in the following screenshot, the **Type** is set as **Area (baked only)** and cannot be changed:



Area lights can be used instead of point lights when baked lighting is okay and you desire software shadows.

To create an area light, we select **GameObject | Light | Area Light** from the top menu.

Implementing reflection probes

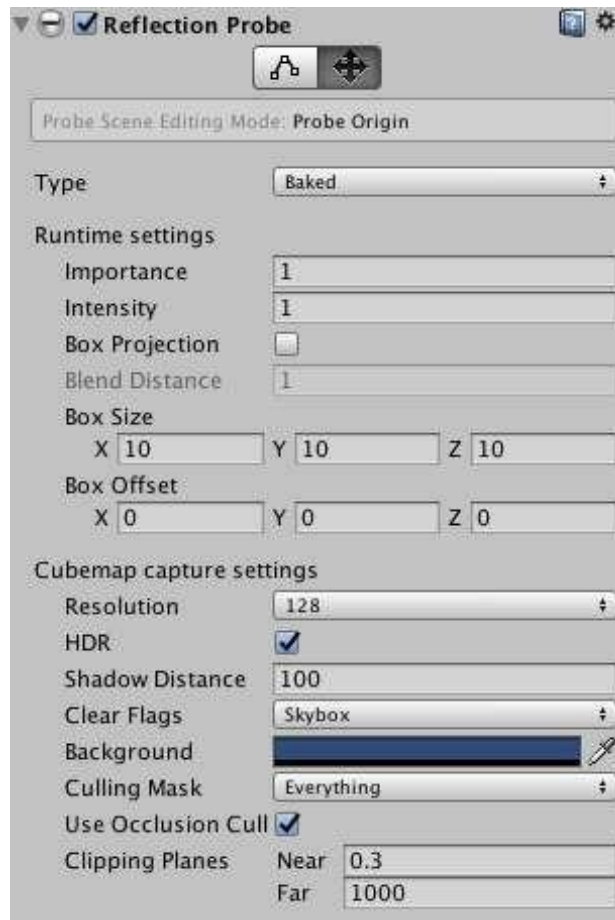
Reflection probes capture a 360° spherical view of their surroundings. In this sense, it is somewhat like a camera. That captured image is used by nearby objects that have reflective materials.

To create a reflection probe, we select **Game Object | Light | Reflection Probe** from the top menu.

As you can see from the following screenshot, the reflection probe is a sphere and has captured the view of its surroundings. When objects are placed in close proximity to the reflection probe, the reflections will be visible on the object:



A review of a reflection probe in the **Inspector** panel, shown as follows, reveals several settings that we can change to affect how the probe works and how it impacts our game environment:



The following list of properties highlights the ones you are most likely to change in the Unity games you create:

Type: You can select **Baked**, **Custom**, or **Realtime**. Remember, we can improve game performance if we bake our lighting whenever possible.

Importance: When there are multiple rendering probes in the area, you can set the importance of each of them. The higher the value, the greater the importance.

Intensity: The lowest value is zero. You can experiment with the results of changing this setting.

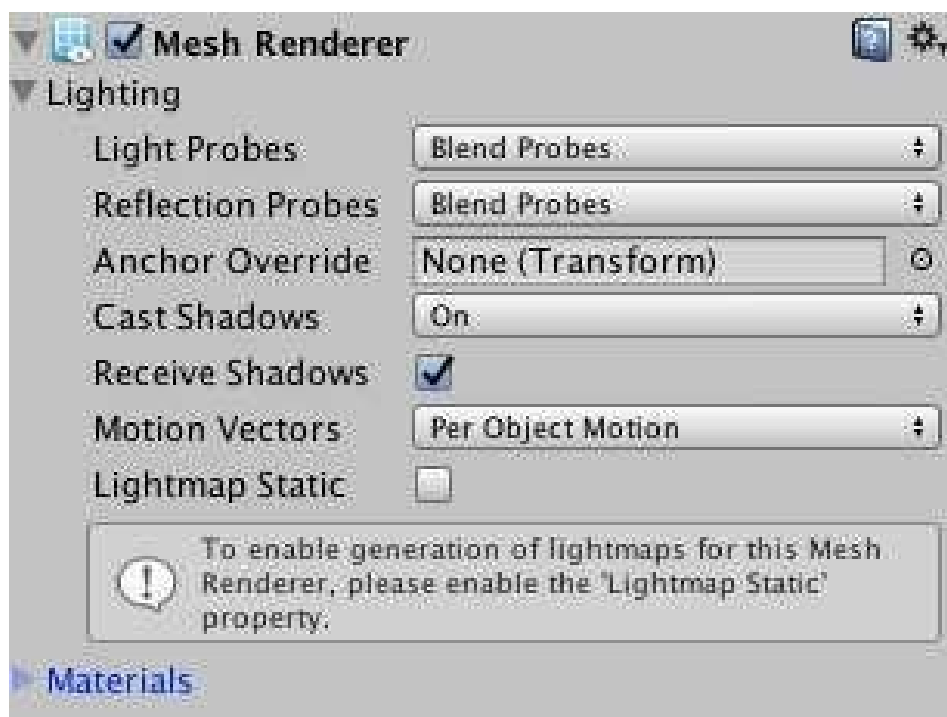
Resolution: You can select **16**, **32**, **64**, **128**, **256**, **1024**, or **2048** for the resolution of the captured image reflection.

Understanding shadows

As indicated earlier in this chapter, our game scenes can have numerous light sources, and we can enable or disable an object's ability to cast or receive shadows. We have shadows in the real world and it is important to consider, for our Unity games, what objects cast shadows and what objects receive shadows.

The following screenshot shows the **Mesh Renderer** component of an object in the

Inspector panel. Let's review the key settings of this component:



- **Light Probes:** It can be set to **Blend Probes**, **Use Proxy Volume**, or **Off**. You will most likely use the default **Blend Probes** for simple Unity games.
- **Reflection Probes:** This setting can be turned off or set to **Blend Probes**, **Blend Probes And Skybox**, or **Simple**.
- **Cast Shadows:** This setting can be set to **On**, **Off**, **Two-Sided**, or **Shadows Only**. The default is **On**, so you should disable this for all objects that do not need to cast shadows.

- **Receive Shadows:** This setting is a toggle that tells Unity whether you want that object to receive shadows or not. As you would expect, this takes extra processing to display during the game. So, if you do not need an object to receive shadows, deselect this for greater performance.

Chapter 6: Basic Scripting with C#

C# Variables

Variables are containers for storing data values.

In C#, there are different **types** of variables (defined with different keywords), for example:

- **int** - stores integers (whole numbers), without decimals, such as 123 .
- **double** - stores floating point numbers, with decimals, such as 10.99.
- **char** - stores single characters, such as 'a' or 'B'. Char values are surrounded by single quotes
- **string** - stores text, such as "Hello World". String values are surrounded by double quotes
- **bool** - stores values with two states: true or false

C# Data Types

A data type specifies the size and type of variable values.

- **Numbers**

Number types are divided into two groups:

- **Integer types** stores whole numbers, positive or negative (such as 123 or -456), without decimals. Valid types are **int** and **long**. Which type you should use, depends on the numeric value.

- **Floating point types** represents numbers with a fractional part, containing one or more decimals. Valid types are **float** and **double**.

- **Integer Types**

- The **int** data type can store whole numbers from -2147483648 to 2147483647. In general, and in our tutorial, the **int** data type is the preferred data type when we create variables with a numeric value.

- **Floating Point Types**

- You should use a floating point type whenever you need a number with a decimal, such as 10.99 or 2.13516.
- The **float** and **double** data types can store fractional numbers.

- **Booleans**

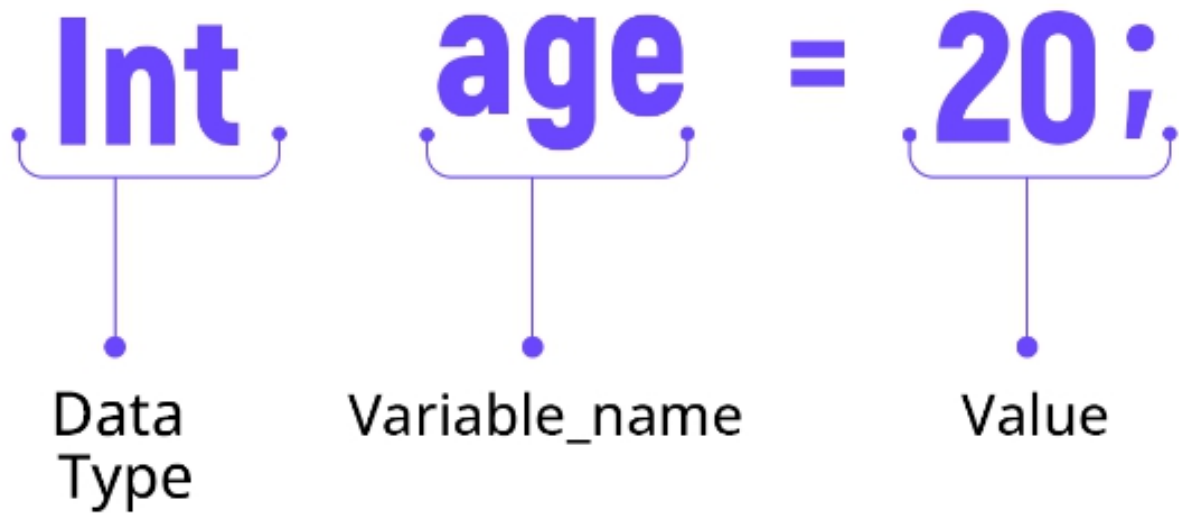
- A boolean data type is declared with the **bool** keyword and can only take the values **true** or **false**.

- **Characters**

- The **char** data type is used to store a **single** character. The character must be surrounded by single quotes, like 'A' or 'c'

- **Strings**

- The **string** data type is used to store a sequence of characters (text). String values must be surrounded by double quotes



Definition Of OOP ' s

- Object-oriented programming is based on the concept of objects. In object-oriented programming data structures, or objects are defined, each with its own properties or attributes. Each object can also contain its own procedures or methods.
- OOP ' s Stands for Object Oriented Programming System.
- OOP ' s is about creating Objects that contain both Data and Methods.

Concept Of OOP ' s

1. Classes and Objects
2. Methods
3. Inheritance
4. Polymorphism
5. Abstraction
6. Encapsulation

- **Classes :-**

1. Classes are blueprint/Template.
2. Class is not a real world Entity.
3. Class do not occupy memory

- **Object :-**

1. Object is an Instance of Class.
2. Object is a real world entity.
3. Object occupy memory.

- ☐ Class has attributes and behaviour
- ☐ Based on class , object has same attributes and behaviour.
- ☐ For Examp

Class	Objects
Car	Ferrari
Attributes:	Attributes:
Model	Model
Color	Color
Price	Price
Behaviour:	Behaviour:
Run()	Run()

Accelerate()	Accelerate()
--------------	--------------

- **Class Syntax :**

□ [access modifier] - [class] - [identifier]

Create a class named " Car " with a variable color :

```
class Car
{
    string color = "red";
}
```

- **Objects Syntax :**

Create an object called " myObj " and use it to print the value of color :

```
class Car
{
    string color = "red";

    static void Main(string[] args)
    {
        Car myObj = new Car();
        Console.WriteLine(myObj.color);
    }
}
```

- **Methods :**

1. A **method** is a block of code which only runs when it is called.
2. You can pass data, known as parameters, into a method.
3. Methods are used to perform certain actions, and they are also known as **functions**.

4. Why use methods? To reuse code: define the code once, and use it many times.
5. For example :-
- 6.

```

Voide Update ()
{
    Movement();
}

Void Movement()
{
    //Code line
}

```

- Recursion Method :-
 1. Recursion is derived from the word recur which means repetition.
 2. Recursion is the process by virtue of which a method can call itself.

3.For example:-

```

Void fun()
{
    Fun();
}

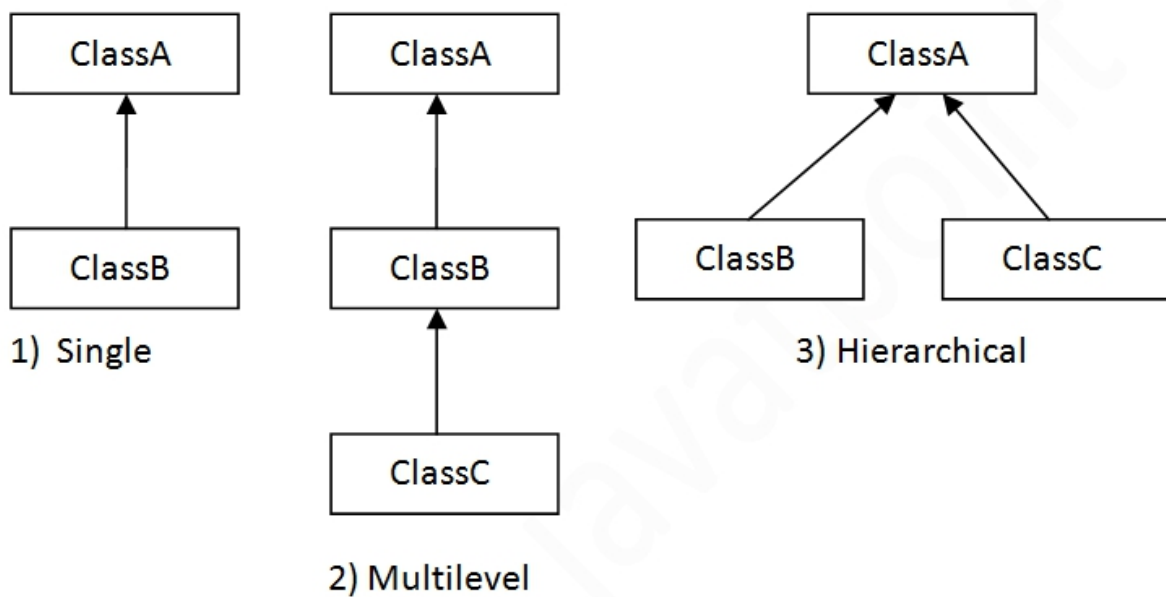
```

• Access Modifier :-

1. Access modifier specify the visibility/accessibility of class and its members.
2. Access modifier provide restriction in class and its members.
3. For example:- Private , Public , Protected.

- **Inheritance: -**

1. Inheritance is a process of inheriting the properties and behaviours of existing class into new class.
2. There are only 3 types of inheritance used in C# :-
 - Single Inheritance.
 - Multilevel Inheritance.
 - Hierarchical Inheritance.



- ☐ Refer the diagram above for simplification of the types of Inheritance.

Syntax Simple Inheritance:-

```
Class <parent_class>
```

```
{
```

```
//Code line...
```

```
}
```

```
Class <child_class>:<Parent_class>
```

```
{  
//Code line....  
}
```

- ☐ Protected class cannot be called in any type of inheritance.

1. **Single Inheritance:-** There is only one parent class and one child class.

Class A

```
{  
.....  
}
```

Class B:A

```
{  
....  
}
```

- ☐ Class A gets inherited in Class B .

2. **Multilevel Inheritance:-** Derived class will inherit a base class and as well as the derived class also act as the base class to other class.

Class A

```
{  
....  
}
```

Class B : A

```
{
```

.....

}

Class C : B

{

.....

}

- ☐ Class A is inherited to Class B and then Class B gets inherited in Class C.

3.Hierarchical Inheritance :-

1. One Parent class that has more than one child class.
2. Hierarchical inheritance describes a situation in which a parent class is inherited by multiple subclasses.

Class A

{

.....

}

Class B1 : A

{

....

}

Class B2 : A

{

.....

}

- ☐ Class A is Inherited in Class B1 as well as in Class B2.

• Polymorphism :-

1. Poly means many , Morphism means Forms/Behaviours
2. Polymorphism means having many forms.
3. It occurs when we have many classes that are related to each other by inheritance.
4. For example :-

```
class Animal // Base class (parent)
{
    public void animalSound()
    {
        Debug.log ("The animal makes a sound");
    }
}
```

```
class Pig : Animal // Derived class (child)
{
    public void animalSound()
    {
        Debug.log ("The pig says: wee wee");
    }
}
```

```
class Dog : Animal // Derived class (child)
{
    public void animalSound()
    {
        Debug.log("The dog says: bow wow");
    }
}
```

There are two sub types of Polymorphism :-

- Compile time polymorphism/ Static Polymorphism.
- Run time Polymorphism / Dynamic Polymorphism.

1. Compile time Polymorphism :- In compile time polymorphism, the compiler identifies which method is being called at the compile time.

It is achieved by using method overloading.

- **Method Overloading :-**

1. Class can have multiple methods having same name .

- Conditions of Method Overloading :-
 1. Method should have same name.
 2. All methods should be in same class.
 3. All methods should have different parameters or different number of parameters.
 4. For eg:-

Class program

```
{  
Void Sum (int a , int b)  
{  
Console.WriteLine(a+b);  
}  
Void Sum (float a , float b)  
{  
Console.WriteLine(a+b);  
}  
Static void Main (string[] args)  
{  
Program obj = new program();  
Obj.Sum(10,20);  
Obj.Sum(10.5f ,20.5f);  
Console.ReadLine();  
}  
}  
}
```

2.RunTime Polymorphism / Dynamic Polymorphism :-

- In Runtime Polymorphism the compiler resolves the object at run time and then it decides which function call should be associated with that object.
- It is achieved by using method overriding.
- **Method Overriding:-** Method Overriding can override the method of parent class with same method of child class.

- Conditions of Method Overriding:-

1. Same name of methods.
2. Methods should be in different class.
3. Should have same parameters
4. There should be inheritance between class
5. For example: -

```
class Animal
{
    public void Eat()
    {
        Console.WriteLine("The animal is eating");
    }
}

class Dog : Animal // Derived class (child)
{
    public void Eat()
    {
        Console.WriteLine ("The Dog is Eating");
    }
}

Public static void Main(string[] args)
{
    Dog tommy=new Dog();
    Tommy.Eat();
    Console.ReadLine();
}
```

- Constructor :

1. Constructors contains the same name as of the class
2. The advantage of a constructor, is that it is called when an object of a class is created.
3. Types: Default , User Defined , Parameterized.

- **Key elements of constructors:**

1. Constructors does not have a return type.
2. Access modifiers can be used with constructors.
3. Constructors invokes when objects get created .
4. Constructors allocates memory to the object.
5. Constructors Initializes the member variable of class .
6. Can declare more than one constructor in class.
7. Constructors can be overloaded .

- **Encapsulation :**

1. Encapsulation means hiding sensitive data.
2. Encapsulation means Wrapping the data (Variables) and code acting on the data (Methods) together as a single unit.
3. By declaring the method with access modifier encapsulation can be achieved.

- **Abstraction :**

1. Abstraction is a process to hide the certain details and show only the functionality.
2. Abstraction can be achieved by either abstract class or interfaces.

- **Abstract Class :-**

1. Abstract Class is a restricted class that cannot be used to create objects.
2. To access the abstract class, it must be inherited from another class. Let's convert the Animal class we used in the [Polymorphism](#) Section to an Abstract class.

// Abstract class

```
abstract class Animal
```

```
{
```

// Abstract method (does not have a body)

```
public abstract void animalSound();
```

```

// Regular method
public void sleep()
{
    Console.WriteLine("Zzz");
}
}

// Derived class (inherit from Animal)
class Pig : Animal
{
    public override void animalSound()
    {
        // The body of animalSound() is provided here
        Console.WriteLine("The pig says: wee wee");
    }
}

class Program
{
    static void Main(string[] args)
    {
        Pig myPig = new Pig(); // Create a Pig object
        myPig.animalSound(); // Call the abstract method
        myPig.sleep(); // Call the regular method
    }
}

```

- Abstract Method :-

1. Abstract Method can only be used in an abstract class.
2. Abstract Method does not have a body .
3. Body get provided by the derived class.

Introduction to C# for Unity

C# is the programming language used in Unity for scripting. It is an object-oriented language, which means it uses objects to represent data and methods to define behavior. C# is known for being easy to learn and powerful enough to handle complex game development tasks.

- **MonoBehaviour**

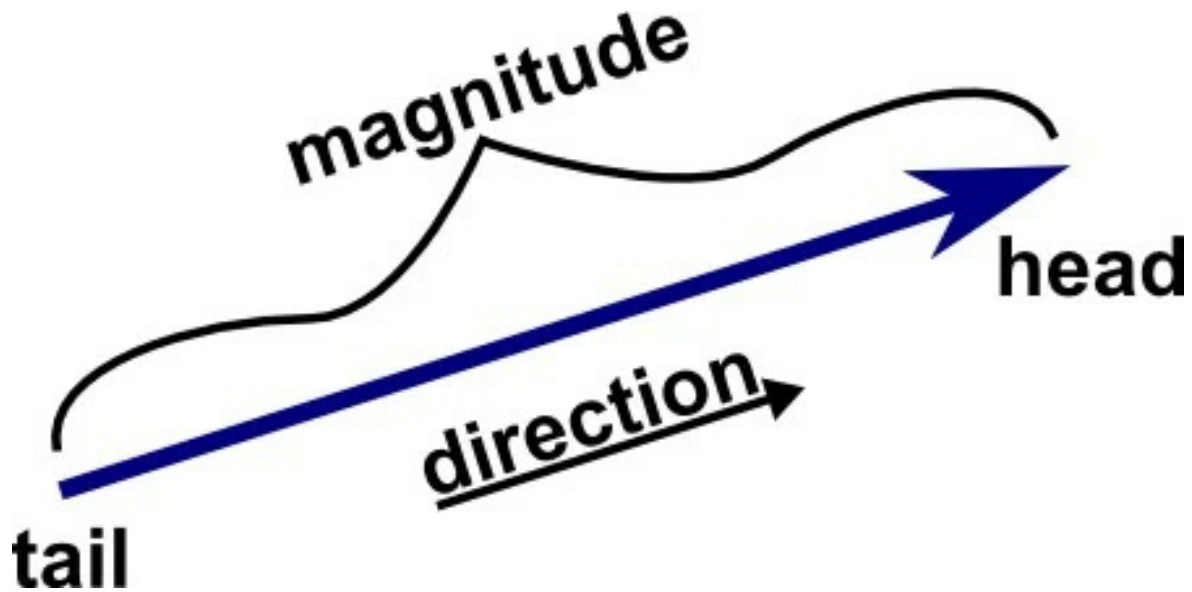
- The MonoBehaviour class is the base class from which every Unity script derives, by default. When you create a C# script from Unity 's project window, it automatically inherits from MonoBehaviour, and provides you with a template script.

- **Vectors in Unity :**

1. Vectors are of 3 types :
 - Vector 2 , Vector 3 and Vector 4 which is represented as Vector 2D , Vector 3D and Vector 4D respectively.
 - However Vector 4D is rarely used as technologies are yet to evolve,

This structure is used in some places to represent four component vectors (e.g. mesh tangents, parameters for shaders).

2. Vectors contains magnitude and direction.
3. In math, a vector is pictured as a directed line segment, whose length is the magnitude of the vector and the arrow is indicating the direction where the vector is going:



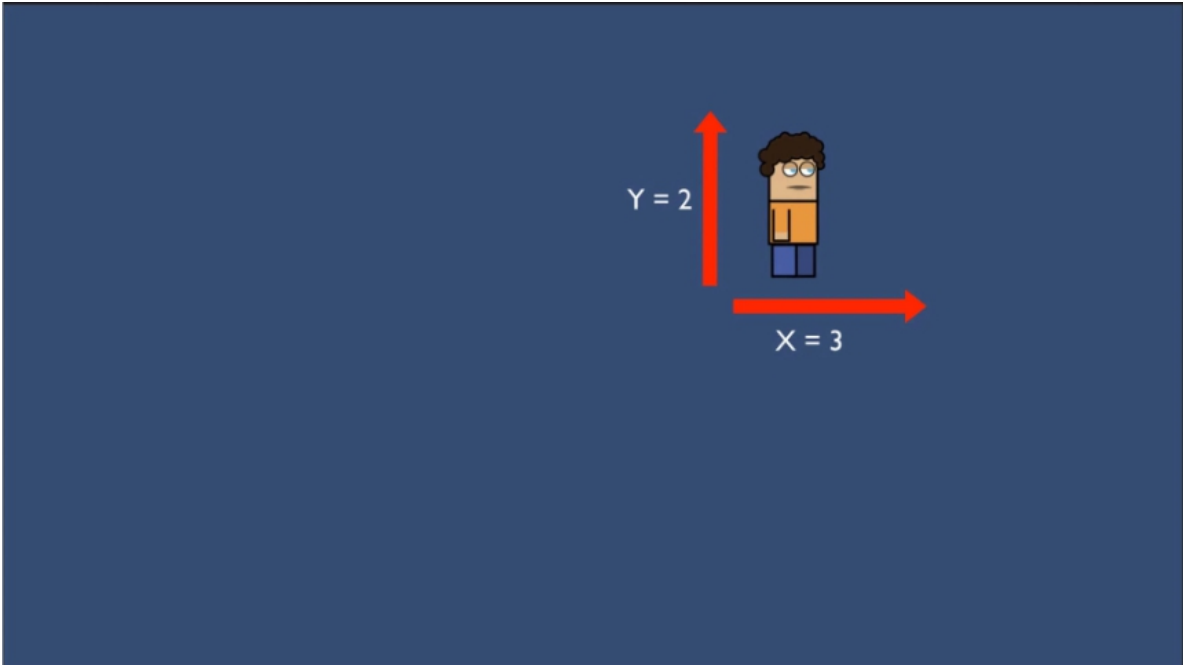
4. In game development, vectors are mainly used to describe the position of a game object, and to determine its speed and direction.

- **Vector2 And Vector3 In Unity**

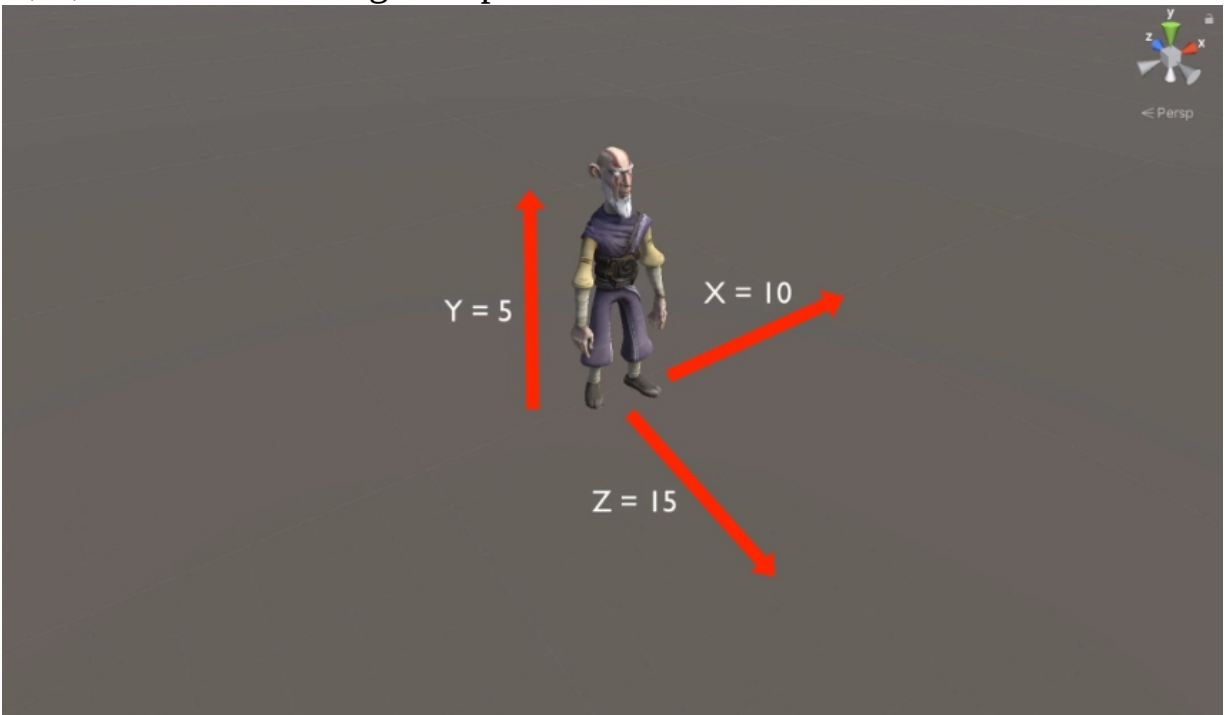
We already mentioned that in game development vectors describe the position of a game object, it's the same with Unity.

A Vector2 object describes the X and Y position of a game object in Unity. Since it's a Vector2, and it only describes X and Y values, it is used for 2D game development.

In a 2D game, if you move a game object with an x amount of units on the X axis, and y amount of units on the Y axis you will get the position of that game object:



The same goes for a 3D game, except that you can move the game object on X, Y, and Z axis in the game space:



The values for the axis are located in the Position property of the Transform component of course:

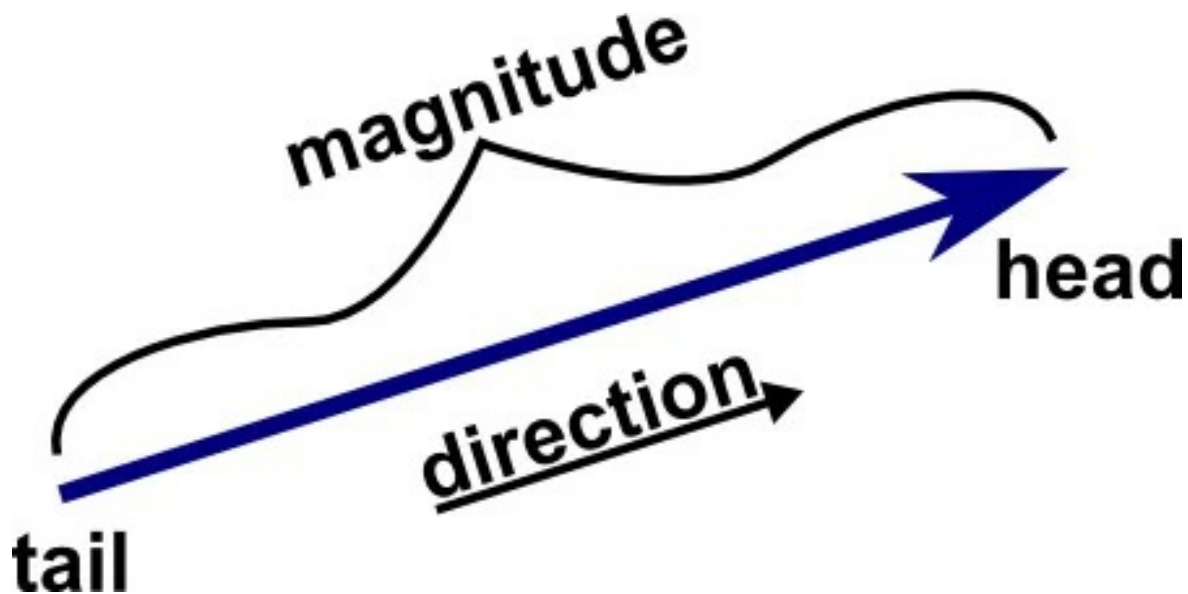


The Magnitude Of The Vector

One of the things we use vectors for is to know the magnitude of the game object.

But what is magnitude of a vector?

The magnitude is the distance between the vectors origin (0, 0, 0) and its end point. If we imagine a vector as a straight line, the magnitude is equal to its length as we saw in the first image:



To calculate the magnitude in math, given a position vector $\vec{v} = \langle a, b \rangle$, the magnitude is found by $\text{magnitude} = \sqrt{a^2 + b^2}$ (square root of a squared + b squared).

In 3D it works the same way, except that we have one more axis in the calculation $\text{magnitude} = \sqrt{a^2 + b^2 + z^2}$ (square root of a squared + b squared + z squared).

But in Unity we simply do this:

```
1 | Vector3 testVector = new Vector3(10, 15, 20);  
2 | float testVectorMagnitude = testVector.magnitude;
```

Calling .magnitude on a vector variable will return the magnitude of that vector. And if you hover over the magnitude word in the code, you will see the explanation which states that magnitude will return the length of the vector, which is what we explained using the image above:

```
P float Vector3.magnitude { get; }  
Returns the length of this vector (Read Only).
```

- For What Do We Use The Magnitude Of A Vector

The magnitude of a vector is used to measure the speed of the vector. I say speed of the vector but it's actually the speed of the game object.

For example, if we are moving the game object using its Transform component, we can limit the movement speed using the magnitude of the vector:

```
private float speedX, speedZ, moveSpeed = 10, maxSpeed = 100;
```

```
Vector3 tempVector;
```

```
void Update()
```

```
{
```

```
    speedX = Input.GetAxisRaw("Horizontal");
```

```
    speedZ = Input.GetAxisRaw("Vertical");
```

```
    tempVector = transform.position;
```

```
    // if the speed of the vector is less than the
```

```

// maximum allowed speed
if (tempVector.magnitude < maxSpeed)
{
    tempVector += new Vector3(speedX, 0f, speedZ) * (moveSpeed *
Time.deltaTime);
}
transform.position = tempVector;
}

```

The same goes for calculating the speed of a rigidbody. Since

```

1 | rigidbody.velocity

```

is a vector, we can get the speed of that rigidbody by calling

```

1 | rigidbody.velocity.magnitude

```

and we can use this information to limit the movement speed of the rigidbody the same we limited the movement speed of the Transform component in the example above.

We can also use the magnitude to calculate the distance between two vectors. If we have two vectors, a and b, then:

```

1 | (Vector A - Vector B).magnitude

```

is the distance between these two vectors. This is what the Vector3.Distance function actually does.

• The Direction Of The Vector

One of the most common informations we need in a game is the direction of vectors which indicates in which direction a specific game object is

going. This is used to move characters in the game, to create enemy AI and so on.

To get the direction of the vector we need to normalize it, and in Unity we normalize a vector like this:

```
1 private Vector3 vectorA;  
2 Vector3 vectorA_Direction = vectorA.normalized;  
3  
4 // or  
5  
6 vectorA.Normalize();
```

Using normalized or Normalize will give us the direction of the given vector.

Now you are probably asking what is the difference between the two?

Both lines of code will normalize(return the direction) the given vector, but using normalized on a vector will return a new version of the same vector that we can store in a new variable, and the original version of the vector will stay the same.

Using the Normalize function however, will normalize the vector is self e.g. it will change the original vector.

If you hover over the normalized word in the code you will see the following explanation:

```
P Vector3 Vector3.normalized { get; }
```

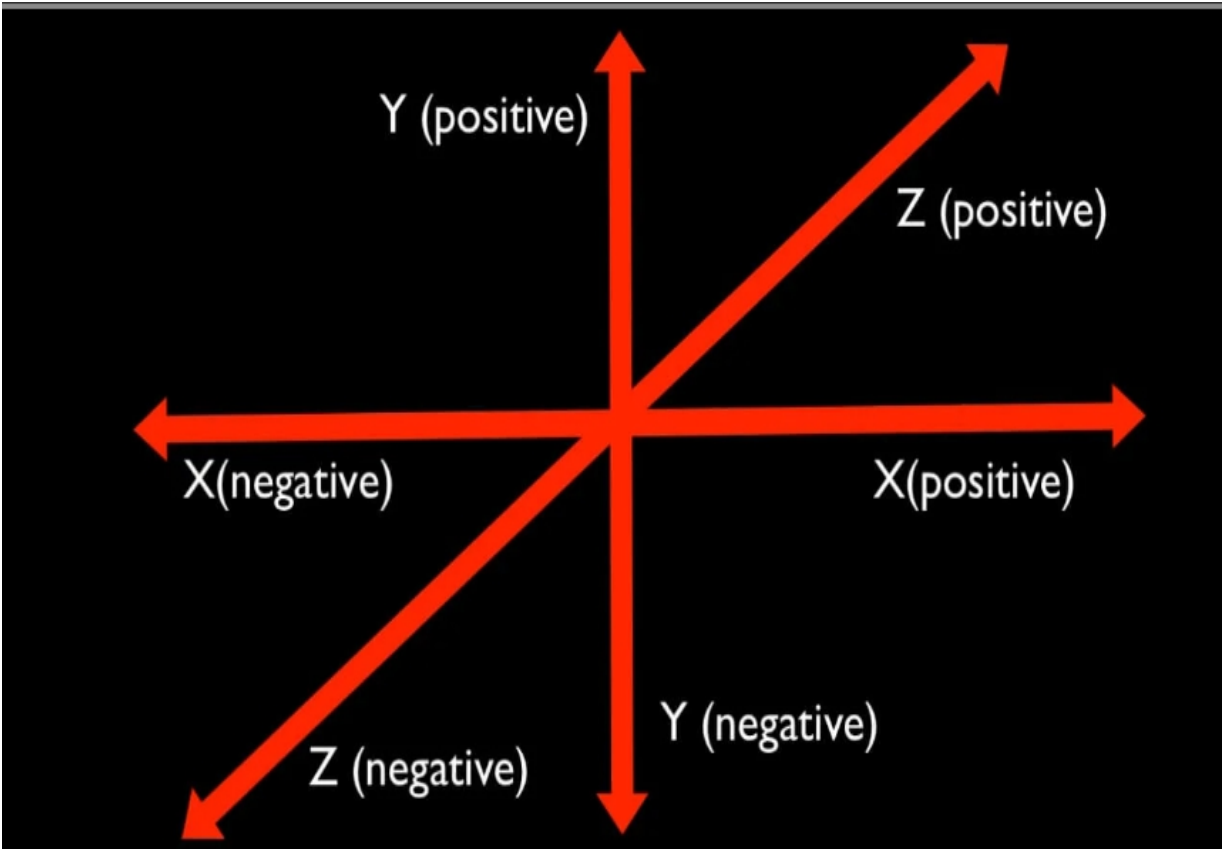
Returns this vector with a magnitude of 1 (Read Only).

This means that the returned vector has a magnitude of 1 e.g. the length of that vector is 1, because this is how directions are represented in Unity.

For example:

```
1 // right direction
2 Vector3 right = new Vector3(1, 0, 0);
3
4 // left direction
5 Vector3 left = new Vector3(-1, 0, 0);
6
7 // up direction
8 Vector3 up = new Vector3(0, 1, 0);
9
10 // down direction
11 Vector3 down = new Vector3(0, -1, 0);
12
13 // forward direction
14 Vector3 forward = new Vector3(0, 0, 1);
15
16 // backward direction
17 Vector3 backward = new Vector3(0, 0, -1);
```

Depending on the axis, adding 1 or -1 value will point to a certain direction. This is connect to Unity ' s coordinate system which looks like this:



As you can see, X is positive on right side and negative on the left, Y is positive up and negative down, and Z is positive forward and negative backwards.

This is why you see the positive 1 and negative 1 values for the vectors in the code example above.

Creating and Attaching Scripts

To create a new script, right-click in the Project window, select "Create," then "C# Script." Give your script a meaningful name, then double-click it to open it in your code editor. To attach the script to a GameObject, drag it from the Project window onto the GameObject in the Hierarchy or use the "Add Component" button in the Inspector.

Basic Scripting Concepts

Scripts in Unity use variables to store data, methods to perform actions, and classes to define objects and their behaviour. For example, you might create a PlayerMovement class that includes a speed variable and an Update method to move the player character based on user input.

```
using UnityEngine;
```

```
public class PlayerMovement : MonoBehaviour
{
    public float speed = 5f;

    void Update()
    {
        float move = Input.GetAxis("Horizontal");
        transform.Translate(move * speed * Time.deltaTime, 0, 0);
    }
}
```

- ### Scene Manager

Unity's scene management system enables developers to efficiently manage game levels, menus, and transitions between different parts of their game.

Following is an example of in script use .

```
1) public void LoadGame()
{
    Scenemanager.Loadscene(1);
}
```

The above script works in increament method where '1' represent increment number of scenes assigned in build settings .

```
2) public void LoadGame()  
{  
Scenemanager.Loadscene("Scene1");  
}
```

The above script works with string value where the name of the scene is written in the brackets with inverted commas.

Chapter 7: Working with Assets Store

Using the Asset Store

The Unity Asset Store offers a wide range of pre-made assets that can speed up your development process. To access the Asset Store, go to Window > Asset Store in the Unity Editor. You can browse, purchase, and download assets directly into your project.

Asset Store packages

Asset Store packages are collections of files and data from Unity projects, or elements of projects.

An **Asset Store** package type is either a **UPM package** or an **asset package** (.unitypackage format). When you add an Asset Store package to your project, the Unity Package Manager unpacks the package and maintains its directory structure and metadata about assets. This metadata includes information such as import settings and links to other assets.

To purchase an Asset Store package from Unity or a third-party publisher, start by searching the Unity Asset Store to find assets that meet your needs. When you find an Asset Store package you want to use, purchase or download it in the Asset Store. You can also make it easier to locate packages you get from the Asset Store by organizing your assets with labels and hiding Asset Store packages that are deprecated or unused.

In the Unity Package Manager, select the My Assets filter to see your list of available Asset Store packages. You can search by name for Asset Store packages. When you find the Asset Store package you want to use, you can add it to your project by using the Package Manager window. For asset packages, refer to Download and import an asset package. For UPM packages, refer to Install a UPM package from Asset Store. If your Asset Store package has a newer version available, you can also update it directly in the Package Manager window. To update an asset package, refer to Updating an asset package. To update a UPM package, refer to Switch to another version of a UPM package.

UPM stands for Unity Package Manager.

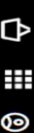
Unity users can become publishers on the Asset Store, and sell the content they have created.

Get popular 3D, GUI, and animation tool assets for just **\$35**. Plus, get an extra **10% off** when you spend \$60 or more with the code **JUNE10OFF**.



AssetStore

Search for assets



3D 2D Add-Ons Audio AI Decentralization Essentials Templates Tools VFX **Sale**

Sell Assets

Over 11,000 five-star assets

Rated by 85,000+ customers

Supported by 100,000+ forum members

Every asset moderated by Unity

EVERYTHING AT \$35

The \$35 Asset sale

Get trending 3D, GUI, environments, and scripting assets for only \$35 and save up to 65%. Plus, get an extra 10% off on orders over \$60 with the code JUNE10OFF.

Shop sale



Chapter 8: Basic Game Mechanics

Creating and Moving GameObjects

To create a new GameObject, go to GameObject > Create Empty. Use the Inspector to add components to your GameObject, such as a Sprite Renderer to display an image or a Rigidbody to enable physics. You can move GameObjects using transform.Translate for basic movement or Rigidbody for physics-based movement.

Handling User Input

Unity's Input class handles user input. Use Input.GetAxis for smooth movement based on input axes like the keyboard or joystick, and Input.GetButton for actions like jumping or shooting.

```
csharp

void Update()
{
    float move = Input.GetAxis("Horizontal");
    transform.Translate(move * speed * Time.deltaTime, 0, 0);
}
```

Basic Physics and Collisions

To add physics to a GameObject, add a Rigidbody component. To detect collisions, add a Collider component, such as BoxCollider or SphereCollider. You can then use methods like OnCollisionEnter to handle collision events.

```
csharp

void OnCollisionEnter(Collision collision)
{
    if (collision.gameObject.tag == "Enemy")
    {
        // Handle collision with enemy
    }
}
```

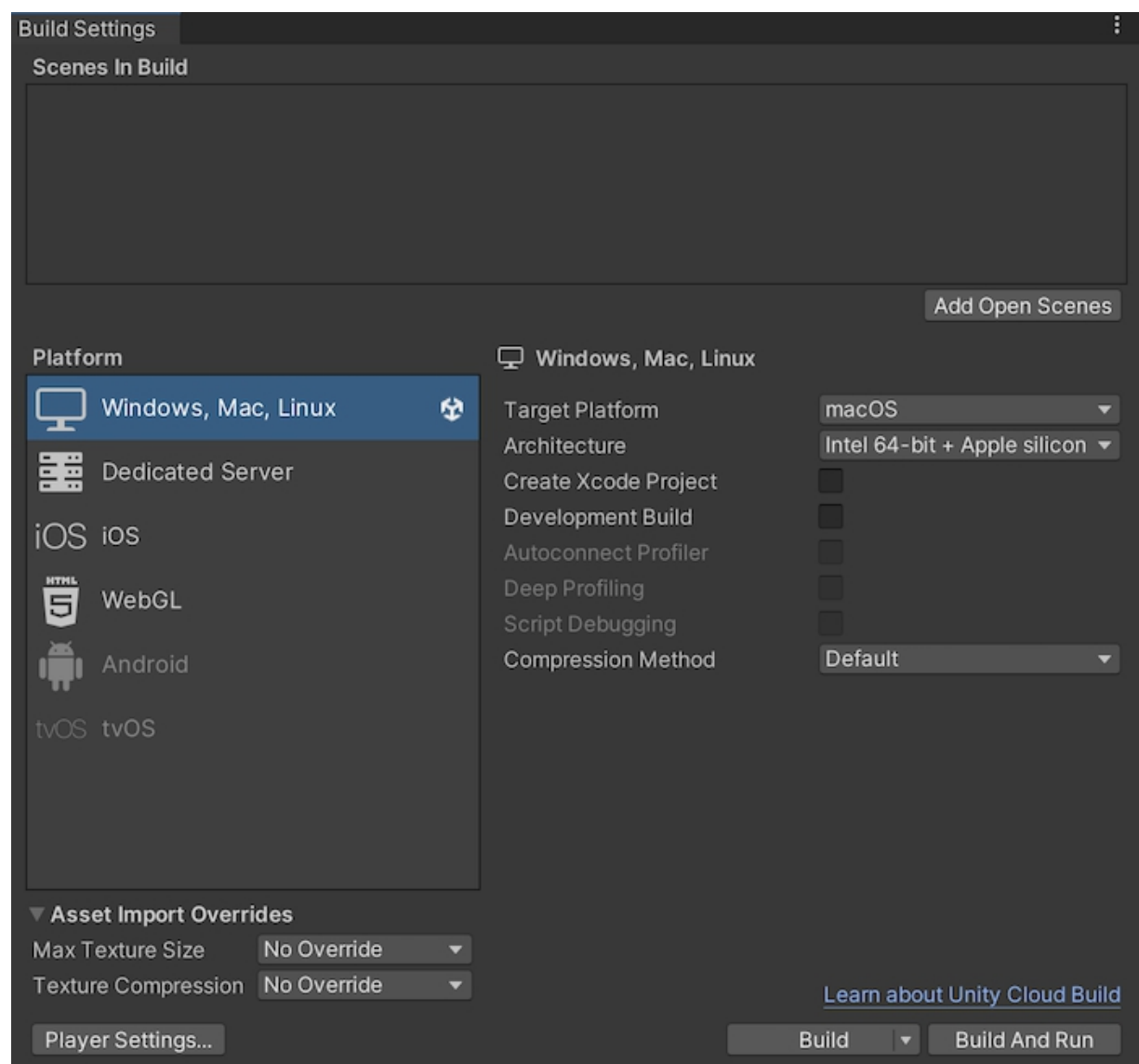
}

}

Chapter 9: Building and Running Your Game

Configuring Build Settings

To configure build settings, go to File > Build Settings. Add your scenes to the build by dragging them into the "Scenes In Build" list. Select your target platform, such as PC or Android.



Generic platform settings

The following settings apply to all platforms.

Setting:	Function:
Development Build	Include scripting debug symbols and the Profiler in your build. You should use this setting when you want to test your application. When you select this option, Unity sets the DEVELOPMENT_BUILD scripting define. Your build then includes preprocessor directives that set DEVELOPMENT_BUILD as a condition
Autoconnect Profiler	Automatically connect the Unity Profiler to your build. This option is only available if you selected Development Build.
Deep Profiling Support	Turn on Deep Profiling in the Profiler. This makes the Profiler instrument every function call in your application so it returns more detailed profiling data. This option might slow down script execution. This option is only available if you selected Development Build.
Script Debugging	Allow your script code to be debugged. This option is only available if you selected Development Build. This option is not available for WebGL.
Wait for Managed Debugger	Make the Player wait for a debugger to be attached before it executes any script code. This option is only available if you selected Script Debugging.

Setting:		Function:
IL2CPP Code Generation		Define how Unity manages IL2CPP code generation. This option is only available if you are using IL2CPP for your scripting backend, not Mono. To change your scripting backend, go to Player Settings > Configuration > Scripting Backend and change from Mono to IL2CPP .
	Faster runtime	Generates code that is optimized for runtime performance. This is the default, and the behavior in previous versions of Unity.
	Faster (smaller) builds	Generates code that is optimized for build size and iteration. It generates less code and produces a smaller build, but may have an impact on runtime performance, especially for generic code. You might want to use this option when faster build times are important, such as when iterating on changes.
Compression Method		Compress the data in your Project when you build the Player. This includes Assets , Scenes , Player settings , and GI data . This option is not available for the WebGL platform.
	Default	On PC, Mac, Linux Standalone, and iOS, there is no default compression. On Android, the default compression is ZIP, which gives slightly better compression results than LZ4HC. However, ZIP data is slower to decompress.
	LZ4	A fast compression format that is useful for development builds.
	LZ4HC	A high compression variant of LZ4 that is slower to build but produces better results for release builds

Building for Different Platforms

There are different platforms to build the game for,

Following are all the platforms that are available in Unity.

- **PC, Mac, and Linux Standalone**
- **Universal Windows Platform**
- **tvOS**
- **iOS**
- **Android**
- **WebGL**

Building your application

- To build your application, select one of the following:
- **Build:** Compile a Player, then do nothing. The default build is incremental, except for the first build, which is always a full non-incremental (clean) build. To force a clean build instead of an incremental build, select Clean Build from the dropdown menu.
- **Build and run:** Compile a Player and open it on your target platform (for more information, see the individual platform pages). This option always uses the incremental build.

Running and Testing Your Game

Use the Play button in the Unity Editor to test your game in Play Mode. The Console window displays logs and debugging information, which can help you identify and fix issues.

Chapter 10: FPS and TPS Concepts

First-Person Shooter (FPS)

In a First-Person Shooter (FPS) game, the player experiences the game from the first-person perspective. This means that the camera is positioned as if the player is looking through the eyes of the character they are controlling. FPS games often focus on shooting and action mechanics, providing an immersive experience where the player feels like they are inside the game world.



Key Features of FPS:

- The camera is positioned at the character's eye level.
- The player's view is typically limited to what the character can see.
- Commonly used for action-packed and immersive gameplay.
- Large 3D segmented game worlds (indoors and outdoors)
Character travels primarily on foot

- Some vehicle usage
- Standard camera and aiming controls Realistic animations
- Large and realistic inventory of hand-held objects (weaponry)
Non-player characters (NPCs) with realistic artificial intelligence Single and multi-player modes

Examples Of FPS Games

This game genre has a long list of successful titles; here are some of them:

- Battlefield
- Borderlands
- Call of Duty
- Half-life

Third-Person Shooter (TPS)

In a Third-Person Shooter (TPS) game, the player views the character they are controlling from a third-person perspective. The camera is usually positioned behind and slightly above the character, providing a wider view of the surroundings. TPS games often combine shooting mechanics with exploration and platforming elements.



Key Features of TPS:

- The camera follows the character from behind.
- The player can see the character they are controlling.
- Offers a broader view of the environment, aiding in navigation and strategy.
- Emphasis on the player character Camera that follows the player
- Player-controlled character motion sequences Full-bodied animations
- Character and camera rotation

Examples Of TPS Games

Here are some of the more successful and popular third-person games:

- Dead Space
- Gears of War
- Resident Evil 2
- Uncharted.

Chapter 11: Next Steps and Resources

Learning Resources

Unity offers a wealth of learning resources to help you improve your skills. Unity Learn provides tutorials and courses, the official documentation is a comprehensive guide to Unity ' s features, and YouTube channels like Brackeys offer video tutorials.

- **Unity Learn:** Unity Learn
- **Official Documentation:** Unity Documentation
- **Tutorials on YouTube:** Channels like Brackeys and Unity's official channel.

Community and Support

The Unity community is a great place to find support and connect with other developers. Join forums, participate in Unity Answers, and engage with Discord communities to get help and share your experiences.

- **Unity Forums:** Unity Forum
- **Unity Answers:** Unity Answers

- **Discord Communities:** Various game dev communities on Discord.

[OceanofPDF.com](https://oceanofpdf.com)



Hariom Mahendra Jagtap

- Technical Artist ,XR Developer,
 - MSc. Animation,
- Assistant Professor : BSc Animation
 - Progressive Education Society's Modern College of Arts, Science and Commerce, Shivajinagar ,Pune 05.

Workshops Conducted :-

AR/VR , IOT and A.I Workshops :

- Modern College of Arts, Science and Commerce, Shivajinagar ,Pune 05.

- Saraswati College of Engineering, Kharghar, Navi Mumbai 410210.
 - Government Polytechnic, Awasari Ambegaon ,412405.-
-
- Karamveer Bhauro Patil College of Engineering, Satara , 415001.

Experience :-

- **3D Generalist** :Immersive Vision Technologies, PVT. LTD, Pune, Maharashtra.
- **XR Developer** : Institute Of Futuristic Technologies ,Mumbai , Maharashtra.
- **XR Developer** : Dream Sculpt Designs , Pune ,Maharashtra.
- **Assistant professor** : Progressive Education Society's Modern College of Arts, Science and Commerce, Shivajinagar ,Pune, Maharashtra.